République Tunisienne
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université de Monastir
Faculté des Sciences

Rapport de Stage de Fin d'Etudes
pour l'obtention de la

Mastère Professionnel : Data Sciences

Sujet : **Système intelligent de détection de fraude (temps réel + explicabilité) : FraudGuard**

Élaboré par : **Ghada Aouissaoui**

Président : **Badreddine Guizani**
Rapporteur : **Samir Belaid**
Encadrant Professionnel : **Chaima Askri**
Encadrant Universitaire : **Mohamed Neji Maatouk**

Effectué à : **Capgemini Engineering Tunisia**

Année Universitaire 2025/2026

Dédicace

Je dédie ce travail à :

À ma mère,

Ton amour inconditionnel, ta douceur, ta patience et tes innombrables sacrifices ont été ma force tout au long de ce parcours. Aucun mot ne pourra exprimer ma gratitude pour tout ce que tu m'as donné. Puisse ce travail être une modeste récompense de tes années de dévouement.

À mon père,

Ton soutien indéfectible, tes conseils avisés et ta confiance en moi m'ont permis de surmonter tous les obstacles. Tu as toujours été un modèle de rigueur et de persévérance. Je te dédie ce travail en signe de mon profond respect et de ma reconnaissance éternelle.

À mes sœurs,

Pour votre présence réconfortante, vos encouragements constants et votre tendresse qui ont illuminé mon chemin. Vous avez toujours cru en moi, même dans les moments de doute. Ce travail est aussi le vôtre.

À mes cousines,

Pour votre soutien affectueux, vos précieux conseils et les moments de partage qui m'ont redonné le sourire. Votre présence à mes côtés m'a donné la force d'avancer. Ce travail vous est également dédié.

À mes amis,

Pour leur fidélité, leur bienveillance et les moments de partage qui m'ont permis de garder le sourire et la motivation. Votre soutien moral a été un véritable moteur.

Ce travail est le fruit de votre amour, de votre confiance et de vos sacrifices.

Je vous en serai éternellement reconnaissante.

Remerciements

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué à la réalisation de ce projet de fin d'études.

Mes remerciements s'adressent tout d'abord à **Monsieur Mohamed Neji Maatouk**, mon encadrant éducatif à la faculté, pour ses orientations pédagogiques, sa rigueur scientifique et ses remarques constructives qui ont grandement enrichi ce rapport.

Je remercie également **Madame Chaima Askri**, mon encadrante professionnelle chez Capgemini Engineering Tunisie, pour sa disponibilité, ses précieux conseils et son accompagnement tout au long de ce projet. Sa confiance et son expertise m'ont permis de mener à bien ce travail dans les meilleures conditions.

Je remercie aussi l'ensemble de l'équipe de Capgemini Engineering Tunisie pour leur accueil et leur soutien, ainsi que les enseignants et le personnel administratif de la faculté pour leur formation de qualité.

Enfin, j'adresse ma reconnaissance aux membres du jury pour le temps qu'ils ont consacré à l'évaluation de ce travail, ainsi qu'à ma famille et à mes proches pour leur encouragement constant tout au long de mes études.

Résumé

La fraude bancaire, en constante évolution, cause des pertes annuelles dépassant 33 milliards de dollars. Face aux limites des systèmes de détection traditionnels, souvent rigides, peu explicables et inadaptés au temps réel, ce projet de fin d'études présente **FraudGuard**, une plateforme intelligente de détection de fraude.

Réalisé au sein de Capgemini Engineering Tunisie, FraudGuard s'appuie sur un pipeline de données robuste (7,4 millions de transactions) et un modèle XGBoost optimisé (AUPRC de 0,957 sur le jeu de test), renforcé par SMOTE pour gérer le déséquilibre des classes. L'explicabilité des décisions est assurée par SHAP, garantissant la conformité réglementaire. La solution est déployée via une API FastAPI dont le temps d'inférence du modèle reste inférieur à 10 ms, pour un temps de réponse complet de l'ordre de 40 ms en usage normal, couplée à un tableau de bord React temps réel et à un agent conversationnel basé sur Groq (Llama 3.3). L'ensemble est conteneurisé avec Docker et supervisé par une stack Prometheus, Grafana et AlertManager.

Ce travail démontre qu'il est possible de concilier haute performance prédictive, explicabilité, temps réel et supervision continue pour répondre aux enjeux opérationnels et réglementaires de la détection de fraude bancaire.

Mots-clés : Détection de fraude, XGBoost, SHAP, FastAPI, Temps réel, DevOps, Explicabilité, Groq, Docker.

Abstract

Bank fraud, constantly evolving, causes annual losses exceeding 33 billion dollars. Traditional detection systems are often rigid, opaque, and ill-suited for real-time processing. To address these limitations, this final year project presents **FraudGuard**, an intelligent fraud detection platform.

Developed at Capgemini Engineering Tunisie, FraudGuard relies on a robust data pipeline (7.4 million transactions) and an optimized XGBoost model (AUPRC of 0.957 on the test set), enhanced with SMOTE to handle class imbalance. Decision explainability is ensured through SHAP, guaranteeing regulatory compliance. The solution is deployed via a FastAPI whose model inference time remains under 10 ms, with an end-to-end response time around 40 ms under normal load, paired with a real-time React dashboard and a conversational agent powered by Groq (Llama 3.3). The entire stack is containerized using Docker and monitored with a Prometheus, Grafana, and AlertManager suite.

This work demonstrates that it is possible to combine high predictive performance, explainability, real-time processing, and continuous monitoring to meet the operational and regulatory challenges of banking fraud detection.

Keywords : Fraud Detection, XGBoost, SHAP, FastAPI, Real-time, DevOps, Explainability, Groq, Docker.

Table des matières

Dédicace i

Remerciements ii

Liste des figures vii

Liste des tableaux viii

Liste des abréviations ix

Introduction 1

1 Cadre général du projet 3
Introduction . . . 3
1.1 Présentation de l'organisme d'accueil : Capgemini Engineering Tunisie . 3
1.2 Présentation du projet . . . 4
1.2.1 Cadre général . . . 4
1.2.2 Problématique . . . 4
1.2.3 Objectifs du projet . . . 4
1.3 Étude et critique de l'existant . . . 5
1.3.1 Solutions existantes . . . 5
1.3.2 Limitations des solutions existantes . . . 6
1.4 Solution proposée : FraudGuard . . . 6
1.5 Méthodologie de travail adoptée . . . 7
1.5.1 Approche de projet . . . 7
1.5.2 Outils de gestion de projet . . . 7
Conclusion . . . 9

2 Analyse et spécification des besoins 10
2.1 Identification des acteurs et diagramme de cas d'utilisation . . . 10
2.2 Besoins fonctionnels . . . 11
2.3 Besoins non fonctionnels . . . 12
2.3.1 Limitation du débit des requêtes . . . 13
2.3.2 Journalisation sécurisée et protection des données personnelles . 14
2.4 Architecture . . . 15
2.4.1 Pipeline de données . . . 15
2.4.2 Architecture système . . . 15
2.5 Environnement logiciel . . . 16
Conclusion . . . 17

3 Données, modélisation et explicabilité 18
3.1 Description du dataset . . . 18
3.2 Analyse exploratoire des données . . . 19
3.2.1 Distribution des transactions . . . 19
3.2.2 Distribution des montants . . . 19
3.2.3 Distribution temporelle de la fraude . . . 20
3.2.4 Corrélation avec la variable cible . . . 20
3.3 Prétraitement et nettoyage des données . . . 21
3.3.1 Audit initial et suppression des doublons . . . 21
3.3.2 Traitement du risque de fuite de données . . . 21
3.3.3 Tri chronologique des transactions . . . 22
3.4 Ingénierie des caractéristiques . . . 22
3.4.1 Variables temporelles . . . 22
3.4.2 Transformation du montant . . . 22
3.4.3 Variables de vélocité . . . 23
3.4.4 Variables cumulatives et historiques client . . . 23
3.5 Découpage temporel et encodage des variables . . . 24
3.5.1 Découpage temporel anti-fuite . . . 24
3.5.2 Encodage des variables catégorielles . . . 24
3.6 Modélisation . . . 24
3.6.1 Rééquilibrage par SMOTE . . . 24
3.6.2 Sélection des features . . . 25
3.6.3 Comparaison des modèles candidats . . . 25
3.7 Synthèse comparative et sélection du modèle . . . 28
3.7.1 Modèle retenu — XGBoost : justification . . . 28
3.7.2 Optimisation des hyperparamètres par recherche en grille . . . 28
3.7.3 Sélection du seuil optimal de décision . . . 29
3.7.4 Évaluation finale sur le jeu de test . . . 29
3.8 Explicabilité du modèle avec SHAP . . . 30
3.8.1 Principe des valeurs de Shapley . . . 30
3.8.2 Importance globale des variables . . . 31
3.8.3 Explication locale d'une transaction . . . 31

4 API et tableau de bord 33
4.1 Architecture du backend . . . 33
4.2 Fonctionnalités exposées par l'API . . . 35
4.2.1 Prédiction du score de risque en temps réel . . . 35
4.2.2 Explication des décisions . . . 36
4.2.3 Simulateur de transactions . . . 36
4.2.4 Diffusion en temps réel . . . 36
4.2.5 Agent conversationnel . . . 36
4.3 Tableau de bord React . . . 37
4.3.1 Choix technologiques . . . 37
4.3.2 Identité visuelle . . . 38
4.3.3 Présentation des pages du tableau de bord . . . 38
4.4 Architecture conteneurisée . . . 45
4.4.1 Exposition réseau et exécution . . . 47
4.4.2 Supervision et observabilité . . . 47

4.4.3 Tests de charge 49

Conclusion générale et Perspectives **51**

Conclusion générale 51

Perspectives 52

Table des figures

1.1 Logo de Capgemini Engineering . . . 3
1.2 Gestion du backlog via GitHub Issues . . . 8
1.3 Tableau Excel de suivi des tâches . . . 8

2.1 Diagramme de cas d'utilisation global – FraudGuard . . . 11
2.2 Pipeline complet de traitement des données FraudGuard . . . 15
2.3 Architecture fonctionnelle de FraudGuard . . . 16

3.1 Distribution des transactions : légitimes vs frauduleuses . . . 19
3.2 Distribution des montants : légitimes vs frauduleuses . . . 20
3.3 Taux de fraude par heure de la journée . . . 20
3.4 Comparaison des métriques de validation avec et sans SMOTE . . . 25
3.5 Stabilité de généralisation du modèle XGBoost. . . . 30
3.6 Graphique en essaim des valeurs SHAP pour la classe « Fraude » . . . 31
3.7 Graphique en cascade SHAP : explication d'une transaction frauduleuse 32

4.1 Architecture du backend FastAPI de FraudGuard . . . 34
4.2 Architecture de l'agent conversationnel FraudGuard. . . . 37
4.3 Page Dashboard : vue principale avec indicateurs temps réel . . . 39
4.4 Page Pipeline de test : import de données et résultats . . . 39
4.5 Page Transactions . . . 40
4.6 Page Clients . . . 40
4.7 Page Alertes . . . 41
4.8 Page Investigation . . . 41
4.9 Page Performance . . . 42
4.10 Page Agent IA . . . 42
4.11 Agent IA — consultation des KPIs globaux . . . 43
4.12 Agent IA — Répartition des décisions . . . 43
4.13 Agent IA — Vue générale clients . . . 44
4.14 Agent IA — investigation d'un profil client . . . 44
4.15 Page Paramètres . . . 45
4.16 Présence des Dockerfiles dans l'arborescence du projet (backend et frontend) . . . 46
4.17 Schéma d'architecture des services conteneurisés et de leurs dépendances 47
4.18 Capture d'écran des conteneurs Docker actifs (VS Code) . . . 47
4.19 Suivi du PSI par variable dans Grafana . . . 48
4.20 Résultats détaillés du test de charge (interface Locust) . . . 49
4.21 Évolution du débit et des temps de réponse pendant la montée en charge 49

Liste des tableaux

1.1 Comparaison entre les solutions existantes et FraudGuard 6

2.1 Identification de l'acteur principal du système FraudGuard 10
2.2 Besoins non fonctionnels du système FraudGuard 13
2.3 Limites de requêtes appliquées par fonctionnalité 14
2.4 Mécanismes de sécurité implémentés 14
2.5 Environnement logiciel du projet FraudGuard 16

3.1 Caractéristiques du dataset de transactions bancaires 18
3.2 Top 10 des variables les plus corrélées avec la cible 21
3.3 Colonnes supprimées pour éviter le data leakage 22
3.4 Récapitulatif des catégories de caractéristiques du modèle 23
3.5 Répartition des données après le découpage temporel 24
3.6 Résultats de la Régression Logistique sur la validation (seuil F1 = 0,6844) 26
3.7 Résultats de LightGBM sur la validation (seuil F1 = 0,5771) 26
3.8 Résultats de XGBoost sur la validation (seuil F1 = 0,6729) 27
3.9 Résultats du Réseau de Neurones sur la validation (seuil F1 = 0,5626) . 27
3.10 Comparaison des quatre modèles sur le jeu de validation 28
3.11 Meilleures combinaisons d'hyperparamètres (recherche en grille) 29
3.12 Règle de décision à quatre niveaux de sévérité 29
3.13 Résultats finaux du modèle XGBoost sur validation et test 29
3.14 Sept premières variables par importance SHAP globale 31

4.1 Exemples de champs caractérisant une transaction 35
4.2 Stack technologique du frontend FraudGuard 37
4.3 Conteneurs Docker du système FraudGuard 45

Liste des abréviations

API	Application Programming Interface
AUPRC	Area Under the Precision-Recall Curve
CSV	Comma-Separated Values
EDA	Exploratory Data Analysis
F1	F1-score
FastAPI	Framework web pour la construction de l'API
FN	False Negative (Faux Négatif)
FP	False Positive (Faux Positif)
Git	Système de contrôle de version
Grafana	Outil de visualisation de métriques
HTTP	HyperText Transfer Protocol
IBM	International Business Machines
JSON	JavaScript Object Notation
KPI	Key Performance Indicator (Indicateur clé de performance)
LLM	Large Language Model (Grand modèle de langage)
ML	Machine Learning (Apprentissage automatique)
MLP	Multi-Layer Perceptron
MUI	Material-UI (bibliothèque de composants React)
PFE	Projet de Fin d'Études
PII	Personally Identifiable Information (Données personnelles)
PostgreSQL	Système de gestion de base de données relationnelle
Prometheus	Système de collecte de métriques
PSI	Population Stability Index
React	Bibliothèque JavaScript pour l'interface utilisateur
Redis	Base de données en mémoire
RGPD	Règlement Général sur la Protection des Données
ROC AUC	Receiver Operating Characteristic - Area Under the Curve
SHAP	SHapley Additive exPlanations
SMOTE	Synthetic Minority Over-sampling Technique
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
SSE	Server-Sent Events
TLS	Transport Layer Security
TN	True Negative (Vrai Négatif)
TP	True Positive (Vrai Positif)
XGBoost	eXtreme Gradient Boosting
PCA	Analyse en Composantes Principales
JWT	JSON Web Token (jeton web JSON)
RBAC	Role-Based Access Control (contrôle d'accès basé sur les rôles)

Introduction

La transformation numérique du secteur financier a profondément modifié les modes de paiement et les comportements transactionnels. L'essor des paiements électroniques, des achats en ligne et des transactions multi-devises offre une flexibilité sans précédent aux consommateurs, mais ouvre également la voie à des fraudes financières toujours plus sophistiquées.En 2022, les pertes mondiales liées aux fraudes par carte bancaire ont dépassé **33 milliards de dollars** [1], une tendance alimentée par la digitalisation croissante et la diversification des canaux transactionnels.

Face à cette menace, les établissements financiers cherchent à automatiser la détection des transactions frauduleuses en temps réel, tout en limitant les faux positifs pour préserver l'expérience des clients légitimes. Les systèmes traditionnels basés sur des règles statiques peinent à s'adapter à l'évolution rapide des schémas de fraude. De plus, les approches d'apprentissage automatique, bien que plus performantes, souffrent souvent d'un défaut d'interprétabilité qui freine leur adoption par les analystes et leur conformité aux exigences réglementaires (RGPD, article 22, droit à l'explication)[2].

Un système de détection de fraude bancaire moderne doit donc concilier quatre exigences fondamentales : une **rapidité de traitement** compatible avec le temps réel, une **précision prédictive** élevée malgré le fort déséquilibre des classes, une **explicabilité** des décisions pour justifier chaque blocage, et une **observabilité** continue en production pour détecter les dérives.

Ce rapport présente **FraudGuard**, une plateforme intelligente de détection de fraude bancaire de bout en bout. Elle repose sur un pipeline complet : ingestion et nettoyage des données, modélisation, explicabilité SHAP, mise en production via FastAPI, et un dashboard analytique interactif avec agent IA conversationnel.

Le document, structuré selon la méthodologie IBM en six phases, s'articule autour de quatre chapitres et d'une conclusion générale. Le premier chapitre pose le cadre général du projet en présentant le contexte industriel, les objectifs et la démarche méthodologique adoptée. Le deuxième chapitre est consacré à l'analyse des besoins fonctionnels et non fonctionnels, ainsi qu'à la conception de l'architecture du système. Les deux chapitres suivants décrivent les réalisations techniques : le troisième aborde l'ensemble de la chaîne de données, de la modélisation et de l'explicabilité du modèle ; le quatrième présente le déploiement de l'API de service, le tableau de bord interactif ainsi que l'architecture conteneurisée de la plateforme.

Enfin, la conclusion générale évalue les résultats obtenus et propose des perspectives d'évolution.

Chapitre 1

Cadre général du projet

Introduction

Ce premier chapitre pose le cadre général de notre projet de fin d'études réalisé au sein de Capgemini Engineering Tunisie. Nous présentons l'organisme d'accueil, la problématique de la détection de fraude financière, les solutions existantes, la solution proposée ainsi que la méthodologie de travail adoptée tout au long du stage.

1.1 Présentation de l'organisme d'accueil : Capgemini Engineering Tunisie

Ce projet a été réalisé au sein de **Capgemini Engineering Tunisie** [4], filiale du groupe Capgemini, un leader mondial de la transformation technologique regroupant 360 000 collaborateurs dans plus de 50 pays. Fondée en juillet 2017, la filiale tunisienne compte plus de 1 500 ingénieurs et professionnels hautement qualifiés, intervenant dans des secteurs stratégiques tels que l'automobile, l'aéronautique, la défense, l'énergie, les télécommunications et l'industrie manufacturière. Ses expertises couvrent l'ingénierie mécanique, électrique et électronique, le développement de logiciels embarqués, la simulation, la modélisation, la cybersécurité et les technologies de l'information.



FIGURE 1.1 – Logo de Capgemini Engineering

1.2 Présentation du projet

1.2.1 Cadre général

Le projet intitulé « FraudGuard : système intelligent de détection de fraude en temps réel avec explicabilité » s'inscrit dans le cadre du projet de fin d'études en vue de l'obtention du diplôme de Master Professionnel en Data Science, au titre de l'année universitaire 2025–2026.

Il vise à concevoir et à développer une plateforme intelligente capable d'identifier, en temps réel, les transactions bancaires potentiellement frauduleuses à l'aide de techniques d'apprentissage automatique. Le système intègre également des mécanismes d'explicabilité afin de justifier les décisions du modèle, ainsi que des outils de supervision en production et d'assistance aux analystes dans l'investigation des cas de fraude.

1.2.2 Problématique

Les institutions financières sont confrontées à un défi majeur : détecter les transactions frauduleuses de manière fiable, rapide et explicable, sans compromettre l'expérience des clients légitimes. Cette problématique est amplifiée par trois contraintes majeures.

La première est le **déséquilibre des classes** : les transactions frauduleuses ne représentent généralement qu'une fraction infime des flux transactionnels (de l'ordre de 1 à 5 % selon les contextes). Les modèles classiques tendent à ignorer la classe minoritaire, produisant des systèmes avec un taux d'*accuracy* trompeur mais une capacité de détection insuffisante.

La deuxième est la **latence** : dans un contexte de paiement en temps réel, la décision d'approuver ou de bloquer une transaction doit être rendue en quelques millisecondes. Toute solution de scoring doit être compatible avec cette contrainte opérationnelle forte.

La troisième est l'**explicabilité réglementaire** : le RGPD (article 22) impose aux institutions financières de pouvoir justifier toute décision automatisée impactant un client. Un modèle de type *boîte noire*, aussi performant soit-il, ne suffit plus sans un mécanisme d'explication locale de ses prédictions.

La problématique centrale de ce projet peut donc se formuler ainsi : Comment concevoir un système de détection de fraude bancaire capable de scorer des transactions en temps réel avec une haute précision, tout en garantissant l'explicabilité des décisions et la supervision continue du modèle en production ?

1.2.3 Objectifs du projet

Ce projet vise à concevoir et déployer **FraudGuard**, une plateforme intelligente de détection de fraude bancaire répondant aux quatre exigences identifiées. Les objectifs

sont les suivants :

— **Construire un pipeline de données robuste** : ingestion, nettoyage, construction des variables et découpage temporel pour garantir l'absence de fuite d'information.

— **Entraîner et évaluer plusieurs modèles** de détection et retenir la solution la plus performante sur les indicateurs métier pertinents.

— **Fournir des explications** locales et globales pour justifier chaque décision du modèle.

— **Déployer le modèle** via une API rapide, avec un temps de réponse compatible avec une utilisation en temps réel.

— **Concevoir un tableau de bord analytique** interactif, permettant de visualiser les transactions et les alertes en flux continu.

— **Mettre en place une supervision continue** de la santé et des performances du système en production, avec des mécanismes de visualisation et de notification.

1.3 Étude et critique de l'existant

1.3.1 Solutions existantes

Deux solutions industrielles majeures dominent actuellement le marché de la détection de fraude bancaire.

FICO Falcon Fraud Manager

FICO Falcon [7] est la solution historique dominante, utilisée par plus de 10 000 institutions financières. Elle repose sur des réseaux de neurones propriétaires et analyse des milliards de transactions en temps réel. Ses principales caractéristiques sont :

— un score transactionnel calculé en temps réel ;

— des modèles comportementaux adaptatifs ;

— l'utilisation de données de consortium (mutualisation des fraudes entre banques) ;

— une interface de gestion des cas (*case management*).

AWS Fraud Detector

AWS Fraud Detector [8] est un service managé d'Amazon (2020) permettant de créer, entraîner et déployer des modèles de détection de fraude sans expertise ML avancée. Il propose un modèle prêt à l'emploi pour la fraude en ligne, une API avec 200 appels par seconde maximum et une intégration native avec l'écosystème AWS.

1.3.2 Limitations des solutions existantes

Malgré leurs atouts, ces deux solutions présentent des lacunes importantes qui justifient le développement d'une solution alternative :

— **FICO Falcon** : coût très élevé, réservé aux grandes banques, modèle propriétaire sans explicabilité locale (pas de SHAP), verrouillage éditeur.

— **AWS Fraud Detector** : dépendance à AWS et arrêt des nouveaux clients depuis novembre 2025, seuil de décision opaque, pas d'explication locale intégrée, logging sans masquage PII automatique.

— **Commun aux deux** : absence d'agent conversationnel en langage naturel, monitoring système non prévu, pas de tests de charge automatisés, gestion basique du déséquilibre des classes.

1.4 Solution proposée : FraudGuard

Notre solution **FraudGuard** répond à l'ensemble des limites identifiées en s'appuyant sur huit composants fonctionnels complémentaires : un moteur de détection par apprentissage automatique, couplé à un mécanisme d'explicabilité pour justifier chaque décision, une API de service fonctionnant en temps réel, associée à une base de données pour conserver l'historique des transactions et des analyses, un tableau de bord interactif qui permet de visualiser les données en flux continu, un assistant conversationnel pour mener des investigations en langage naturel, une stack de supervision complète pour assurer l'observabilité en production, et des dispositifs de sécurité garantissant la protection des données personnelles ainsi que la résilience du service face aux usages abusifs.

Cette solution permet de dépasser les limitations des systèmes existants sur plusieurs points clés. Le tableau 1.1 synthétise les différences clés entre les systèmes standards et FraudGuard.

Tableau 1.1 – Comparaison entre les solutions existantes et FraudGuard

Critère	**Solutions standards**	**FraudGuard**
Explicabilité des décisions	×	✓
Supervision et observabilité continue	×	✓
Assistant conversationnel	×	✓
Protection des données et résilience	Partiel	✓
Validation des performances sous charge	×	✓
Adaptation aux classes déséquilibrées	Partiel	✓
Ouverture et personnalisation	×	✓

1.5 Méthodologie de travail adoptée

1.5.1 Approche de projet

Pour structurer ce projet, nous avons adopté une méthodologie itérative inspirée des pratiques IBM de gestion de projets de data science [9]. Cette approche combine la flexibilité des méthodes agiles avec la rigueur des processus industriels.

Le projet est découpé en six phases successives :

— Phase 1 : Préparation et exploration des données,

— Phase 2 : Modélisation et sélection de la solution,

— Phase 3 : Mise en place de l'explicabilité,

— Phase 4 : Développement du service de scoring,

— Phase 5 : Réalisation de l'interface utilisateur,

— Phase 6 : Sécurisation, tests et Conteneurisation.

Chaque phase est validée avant de passer à la suivante, avec la possibilité de revenir en arrière si nécessaire (par exemple pour corriger une anomalie détectée dans les données). Des réunions hebdomadaires avec le tuteur assurent le suivi de l'avancement et l'ajustement des priorités.

1.5.2 Outils de gestion de projet

Le suivi des tâches est assuré par trois outils complémentaires :

— **GitHub Issues** : gestion du backlog, assignation et suivi de l'état des tâches.

— **Tableau Excel** : suivi détaillé des tâches (identifiant, description, statut, dates, temps passé).

— **Réunions hebdomadaires** : revue des livrables, identification des blocages et validation des choix.

La figure 1.2 illustre le tableau de bord GitHub Issues, où chaque tâche est identifiée par son titre, son assignation et son état d'avancement.

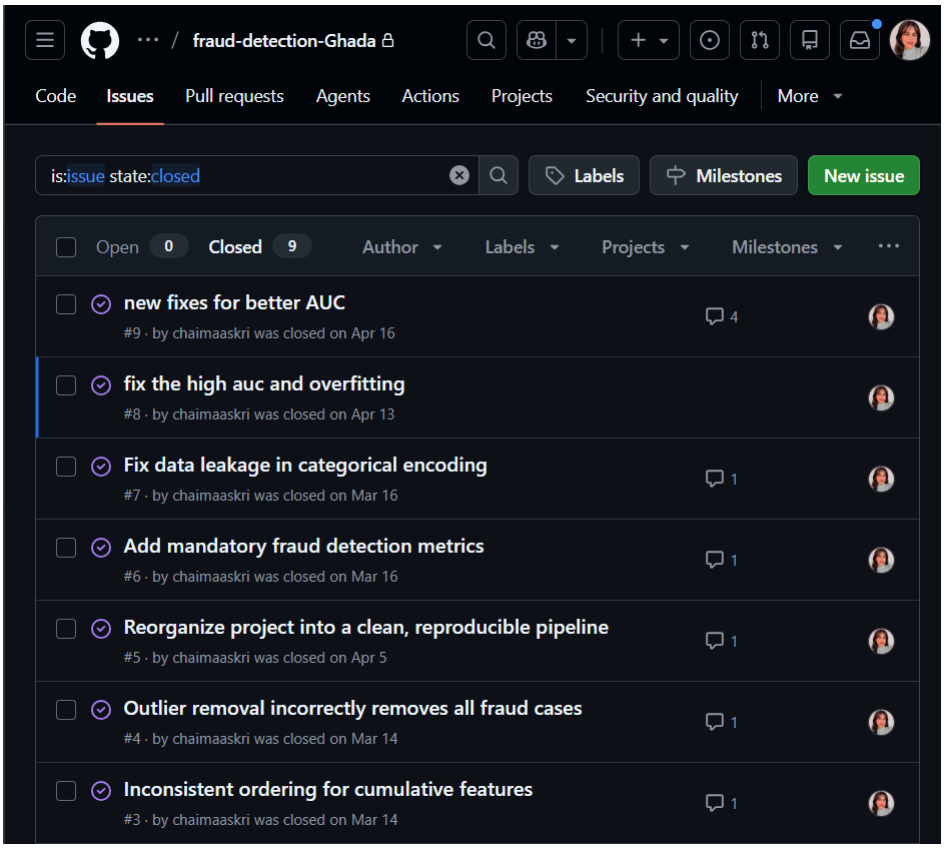


FIGURE 1.2 – Gestion du backlog via GitHub Issues

La figure 1.3 présente le tableau Excel de suivi, qui répertorie l'ensemble des tâches avec leurs statuts, leurs dates de début et de clôture, ainsi que le temps passé.

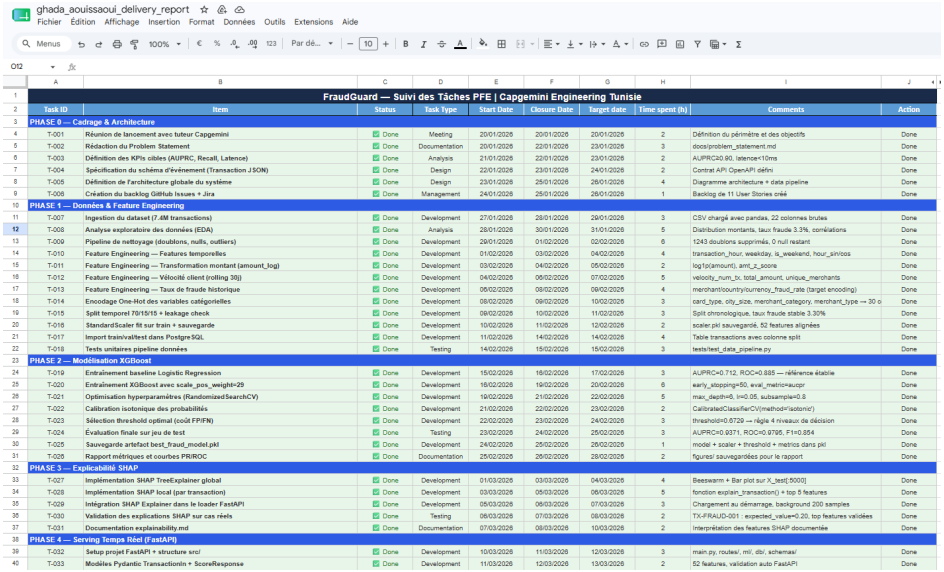


FIGURE 1.3 – Tableau Excel de suivi des tâches

Conclusion

Ce chapitre a présenté Capgemini Engineering Tunisie, le contexte de la fraude bancaire et les limites des solutions existantes qui motivent notre projet. La solution FraudGuard proposée répond à ces lacunes en combinant modélisation performante, explicabilité, serving temps réel et observabilité complète. La méthodologie IBM adoptée, appuyée par GitHub Issues et des réunions hebdomadaires, garantit une gestion rigoureuse et traçable. Le chapitre suivant détaille l'analyse et la spécification des besoins du système.

Chapitre 2

Analyse et spécification des besoins

Introduction

Ce chapitre présente l'analyse des besoins du système FraudGuard. Nous identifions les acteurs impliqués, formulons les besoins fonctionnels et non fonctionnels à un niveau général, puis proposons une vue d'ensemble de l'architecture fonctionnelle du système. Les choix techniques et l'implémentation retenue pour répondre à ces besoins sont détaillés dans les chapitres suivants.

2.1 Identification des acteurs et diagramme de cas d'utilisation

Le système FraudGuard s'adresse principalement à l'analyste fraude, acteur métier chargé de superviser l'activité transactionnelle et d'exploiter les résultats produits par le système. Le tableau 2.1 décrit son rôle.

Acteur	**Rôle**
Analyste fraude	Consulte le tableau de bord en temps réel, analyse les alertes, explore les explications SHAP des décisions, consulte l'historique des transactions et des clients, et interroge l'agent conversationnel pour investiguer des patterns de fraude.

TABLEAU 2.1 – Identification de l'acteur principal du système FraudGuard

Le diagramme de la figure 2.1 présente les principales interactions entre l'analyste fraude et le système FraudGuard. L'analyste peut démarrer une simulation de transactions et observer les résultats en direct, lancer une investigation sur une transaction donnée afin d'obtenir l'explication SHAP associée à la décision du modèle, consulter les alertes générées par la plateforme, consulter l'historique des transactions ainsi que les profils clients, et tester le pipeline de la plateforme en soumettant un nouveau jeu de données. Le scoring des transactions n'apparaît pas comme une action directement pilotée par l'analyste : il s'agit d'un traitement interne du système, déclenché automatiquement lors de la simulation, de la soumission d'une transaction ou de l'évaluation d'un jeu de données via le pipeline de test.

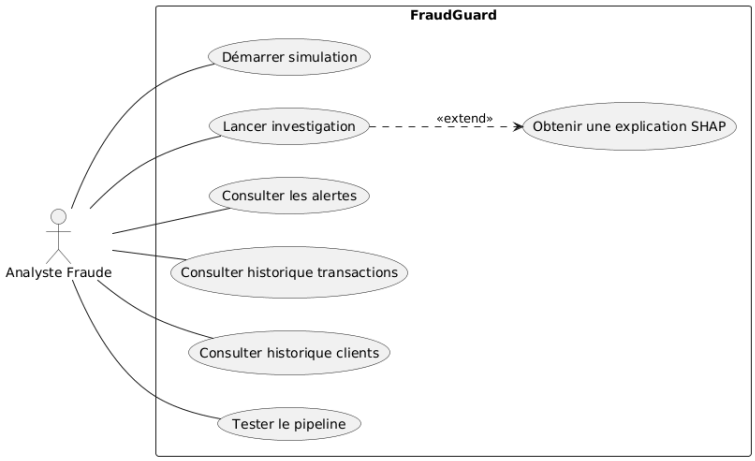


FIGURE 2.1 – Diagramme de cas d'utilisation global – FraudGuard

2.2 Besoins fonctionnels

Les besoins fonctionnels décrivent les actions que l'analyste fraude peut réaliser avec la plateforme. Ils sont organisés par module fonctionnel.

Module Évaluation du risque et prédiction

- **BF1** : L'analyste peut soumettre une transaction et obtenir une prédiction du score de risque ainsi qu'une décision automatique parmi quatre niveaux : approbation, surveillance, révision manuelle ou blocage immédiat.
- **BF2** : L'analyste peut consulter le détail d'une transaction évaluée, incluant le score prédit, la décision, la sévérité et la latence de traitement.

Module Explicabilité

- **BF3** : L'analyste peut obtenir une explication de la décision prise pour une transaction donnée, sous la forme des contributions des principales variables au score calculé.
- **BF4** : L'analyste peut visualiser une vue globale des variables les plus influentes sur l'ensemble des transactions traitées.

Module Tableau de bord

- **BF5** : L'analyste peut consulter un tableau de bord affichant en temps réel les indicateurs clés : volume de transactions, taux de fraude, répartition des décisions et latence moyenne.
- **BF6** : L'analyste peut consulter l'historique des transactions avec filtrage par décision et par sévérité.

Module Gestion des alertes

- **BF7** : L'analyste peut consulter la liste des alertes générées automatiquement par la plateforme pour les transactions à haut risque.

— **BF8** : L'analyste peut résoudre une alerte active en la marquant comme traitée.

Module Investigation et clients

— **BF9** : L'analyste peut interroger la plateforme en langage naturel pour obtenir des analyses sur les tendances de fraude, les transactions à haut risque ou les performances du modèle.

— **BF10** : L'analyste peut consulter le profil d'un client et l'historique de ses transactions avec leurs statistiques agrégées.

Module Pipeline et évaluation

— **BF11** : L'analyste peut soumettre un nouveau jeu de données au format CSV pour obtenir les prédictions de la plateforme sur ces données et, si un label est présent, les métriques d'évaluation associées.

— **BF12** : L'analyste peut démarrer et arrêter un simulateur de transactions, observer en temps réel les scores prédits, les décisions et les statistiques de fraude générées au fil de la simulation.

2.3 Besoins non fonctionnels

Les besoins non fonctionnels définissent les contraintes de qualité que le système doit respecter, indépendamment des fonctionnalités offertes. Le tableau 2.2 présente l'ensemble des besoins non fonctionnels identifiés pour le système FraudGuard.

Tableau 2.2 – Besoins non fonctionnels du système FraudGuard

Code	**Catégorie**	**Description**
BNF1	Performance	Le temps d'inférence du modèle doit être inférieur à 10 ms au p95. Le temps de réponse complet de l'API (avec SHAP et persistance) reste de l'ordre de la seconde sous forte charge (section 4.4.3).
BNF2	Disponibilité	Le backend doit être disponible en permanence, avec un endpoint de vérification de santé permettant la supervision continue.
BNF3	Sécurité	Les données sensibles doivent être hachées avant stockage et masquées dans les journaux. L'accès à l'API est limité par un mécanisme de contrôle du débit afin de prévenir les usages abusifs.
BNF4	Traçabilité	Chaque transaction doit être enregistrée avec l'ensemble de ses métadonnées afin de garantir l'auditabilité complète du système.
BNF5	Explicabilité	Toute décision de blocage doit pouvoir être justifiée par les contributions SHAP des variables, accessibles via l'API et le tableau de bord.
BNF6	Conformité RGPD	Le traitement des données personnelles doit respecter les principes de minimisation et d'anonymisation définis par le règlement général sur la protection des données.
BNF7	Maintenabilité	Le code doit être structuré de manière modulaire et couvert par des tests unitaires et d'intégration.
BNF8	Interopérabilité	L'API REST doit respecter la spécification OpenAPI 3.0 et être documentée automatiquement via Swagger UI.

2.3.1 Limitation du débit des requêtes

Afin de prévenir les abus et de garantir la stabilité du service (BNF3), chaque fonctionnalité exposée par l'API dispose d'une limite de requêtes par minute, adaptée

à sa criticité et à son coût de calcul.

Tableau 2.3 – Limites de requêtes appliquées par fonctionnalité

Fonctionnalité	**Limite**
Scoring d'une transaction	200 requêtes/minute
Explication d'une décision	50 requêtes/minute
Agent conversationnel	30 requêtes/minute
Évaluation d'un nouveau jeu de données	5 requêtes/minute
Contrôle du simulateur	10 requêtes/minute

Lorsqu'un client dépasse la limite allouée, le système renvoie une réponse HTTP 429 l'invitant à réessayer après un court délai, plutôt que de saturer silencieusement le service.

2.3.2 Journalisation sécurisée et protection des données personnelles

Les journaux système sont produits dans un format structuré (JSON) pour faciliter leur exploitation par les outils de supervision. Conformément au RGPD (BNF6), plusieurs mesures sont appliquées :

— Les informations sensibles (emails, noms, numéros de carte, adresses IP) sont automatiquement masquées dans les journaux.

— Les noms et emails des clients sont hachés (SHA-256 avec sel) avant d'être persistés en base de données.

— Lorsque ces informations sont exposées via l'API, seuls quelques caractères sont conservés (exemple : `51***f1`), permettant aux analystes de distinguer les clients sans les identifier.

Tableau 2.4 – Mécanismes de sécurité implémentés

Mécanisme	**Description**
Limitation du débit	Protection contre les usages abusifs et la saturation du service
Hachage des données client	Anonymisation irréversible des informations identifiantes
Masquage dans les journaux	Protection des données personnelles dans les logs et les réponses API
Journalisation structurée	Traçabilité complète exploitable par les outils de supervision
Validation des entrées	Vérification systématique du format et des contraintes des données reçues

2.4 Architecture

2.4.1 Pipeline de données

Le pipeline de données de FraudGuard est organisé en trois grandes étapes. La première consiste à préparer les données brutes issues des transactions. La deuxième regroupe l'ensemble des traitements de préprocessing, notamment le nettoyage des données, le feature engineering, le split temporel des jeux d'entraînement, de validation et de test, ainsi que la normalisation des variables. Enfin, la troisième étape correspond à la modélisation, où les données préparées sont utilisées pour entraîner le modèle avant sa sauvegarde pour une utilisation lors des phases de prédiction et de déploiement. Cette organisation garantit un pipeline clair, reproductible et limitant les risques de fuite d'information (data leakage).

La figure 2.2 illustre ces étapes, depuis les données brutes jusqu'à la phase d'entraînement du modèle.

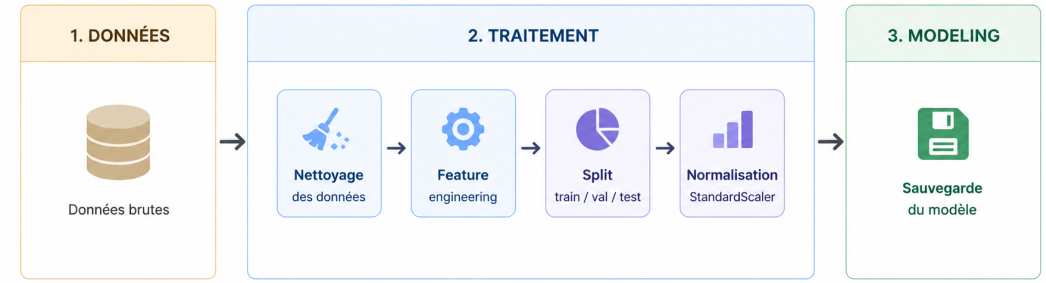


FIGURE 2.2 – Pipeline complet de traitement des données FraudGuard

2.4.2 Architecture système

La plateforme FraudGuard repose sur une architecture en couches, dont la figure 2.3 présente les composants principaux et leurs interactions. Le tableau de bord constitue le point d'accès principal des utilisateurs et communique avec l'API Backend sécurisée. Cette dernière orchestre les services métier, notamment le moteur de détection de fraude, le module d'explicabilité SHAP et l'agent IA conversationnel.

Les performances et le comportement du système sont surveillés en continu par la couche d'observabilité (métriques, supervision, alertes). Parallèlement, l'ensemble des données, des prédictions et des explications est persisté dans la couche de stockage. Cette organisation en couches favorise la modularité, la maintenabilité et l'évolutivité de la solution.

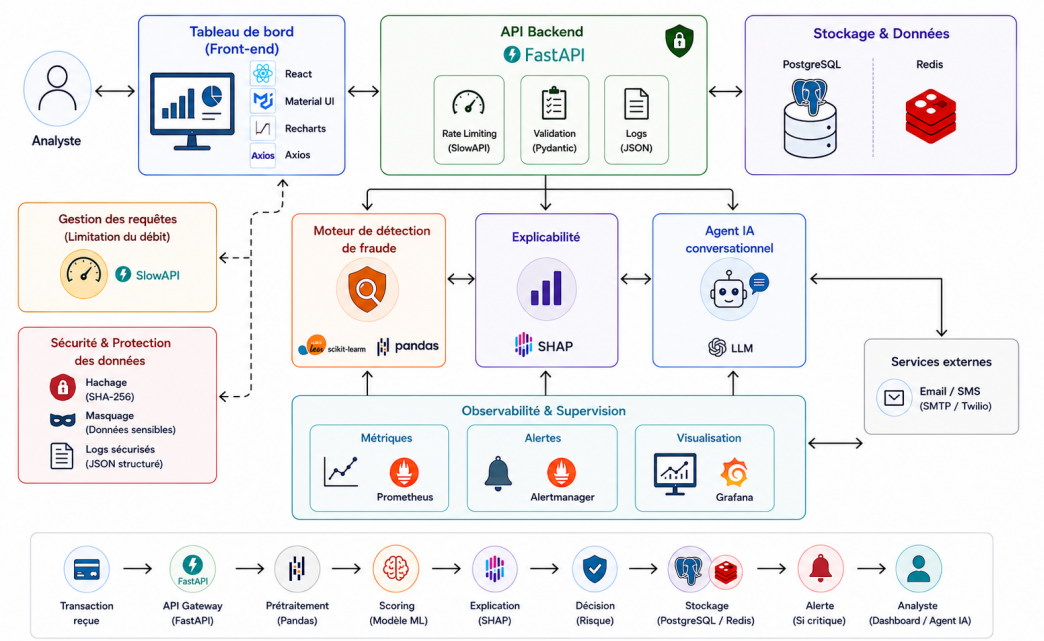


FIGURE 2.3 – Architecture fonctionnelle de FraudGuard

2.5 Environnement logiciel

Le tableau 2.5 présente les principaux logiciels utilisés pour le développement, les tests, le déploiement et la supervision de la plateforme FraudGuard.

TABLEAU 2.5 – Environnement logiciel du projet FraudGuard

Catégorie	**Logiciel**	**Version**
Développement	Visual Studio Code	1.89
	Jupyter Notebook	7.0
	Postman	11.0
Gestion de versions	Git	2.44
Conteneurisation	Docker	26.0
	Docker Compose	2.27
Base de données	PostgreSQL	16
	pgAdmin 4	8.x
Supervision	Prometheus	2.48.0
	Grafana	10.2.0
	Locust	2.24
Serveur applicatif	Uvicorn	0.30.1
	Redis	7.x

Conclusion

Ce chapitre a présenté les besoins fonctionnels et non fonctionnels du système FraudGuard, ainsi que son architecture fonctionnelle, son pipeline de traitement des données et son environnement logiciel. Ces éléments constituent la base de la conception de la plateforme. Le chapitre suivant détaille la phase de préparation des données, de modélisation et d'explicabilité.

Chapitre 3

Données, modélisation et explicabilité

Introduction

Ce chapitre couvre l'ensemble du travail réalisé sur les données du projet FraudGuard, de la description et de l'exploration du dataset jusqu'à la sélection du modèle de détection final, en passant par le prétraitement, l'ingénierie des caractéristiques et l'explicabilité des décisions par la méthode SHAP.

3.1 Description du dataset

Le dataset utilisé est un jeu de données synthétique de transactions financières, généré spécifiquement pour le développement et l'évaluation de modèles de détection de fraude [6]. Conçu pour simuler des patterns réalistes, il couvre un large spectre de catégories transactionnelles : retail (en ligne et en magasin), épicerie, restauration, divertissement, santé, éducation, carburant et voyage. Il présente également une forte diversité géographique et démographique (multi-pays, multi-devises), ainsi que des profils clients détaillés incluant l'ancienneté du compte, les appareils préférés et la plage de dépenses typique.

Le tableau 3.1 résume les principales caractéristiques du dataset.

Tableau 3.1 – Caractéristiques du dataset de transactions bancaires

Attribut	**Valeur**
Nombre total de transactions	7 483 766
Nombre de variables brutes	23 colonnes et une étiquette de fraude
Transactions frauduleuses	environ 1 494 719 (19,97 %)
Transactions légitimes	environ 5 989 047 (80,03 %)
Catégories couvertes	retail, épicerie, voyage, santé, divertissement, etc.
Diversité géographique	multi-pays, multi-devises
Format	fichier texte structuré (CSV), environ 2,93 Go

Ce choix d'un dataset synthétique se justifie au regard des contraintes et des objectifs du projet.

Le recours à un dataset synthétique plutôt qu'à un jeu de données réel, tel que le Kaggle Credit Card Fraud Dataset (ULB, 2013) ou le IEEE-CIS Fraud Detection Dataset (2019), s'explique par plusieurs considérations complémentaires. D'une part, les datasets réels publics présentent des limitations importantes pour ce projet : le dataset ULB ne contient que des features PCA(Analyse en Composantes Principales) anonymisées (V1–V28), ce qui empêche toute interprétation métier des variables et rend l'explicabilité SHAP peu informative pour un analyste fraude. Le dataset IEEE-CIS, bien que plus riche, impose des contraintes de confidentialité strictes sur son usage. D'autre part, le dataset synthétique utilisé offre une diversité transactionnelle réaliste couvrant plusieurs catégories marchands, devises et géographies, tout en permettant de construire des features métier interprétables, conditions nécessaires à la démonstration complète du pipeline d'explicabilité SHAP.

Les données synthétiques, exemptes de bruit de mesure, conduisent naturellement à des métriques plus élevées que sur des jeux de données réels.

3.2 Analyse exploratoire des données

3.2.1 Distribution des transactions

La figure 3.1 présente la répartition du nombre de transactions par classe et confirme le fort déséquilibre du jeu de données.

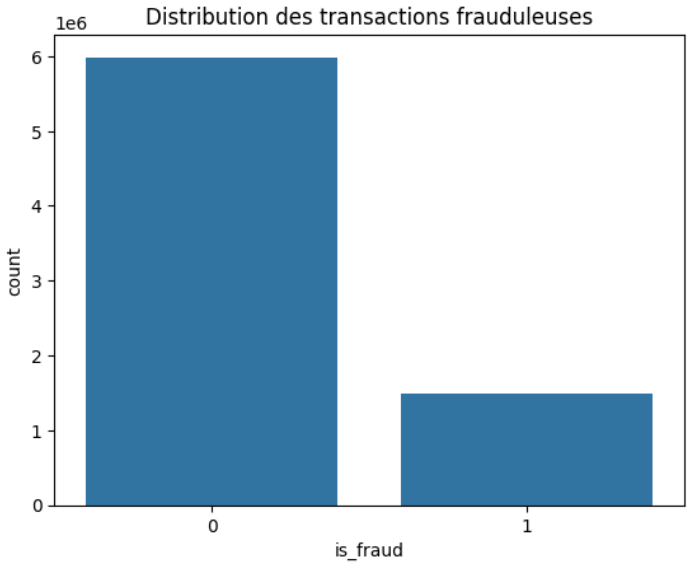


FIGURE 3.1 – Distribution des transactions : légitimes vs frauduleuses

3.2.2 Distribution des montants

La figure 3.2 compare la distribution des montants des transactions légitimes et frauduleuses.

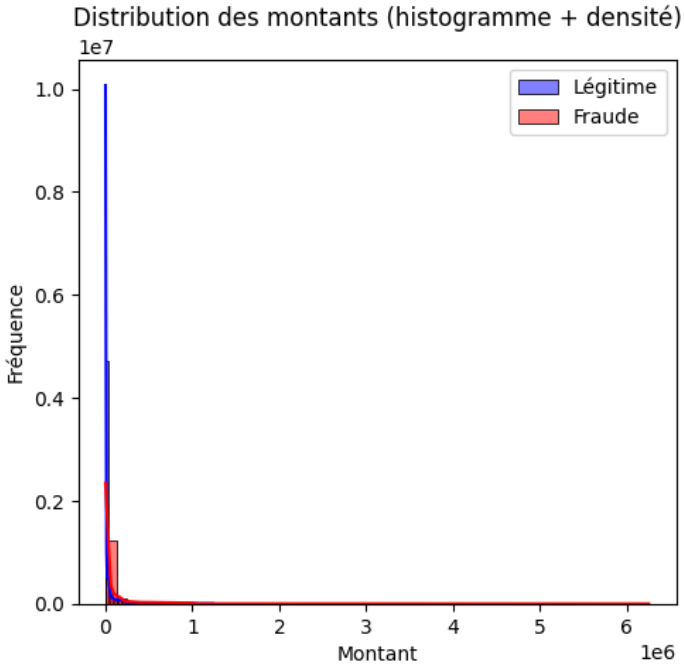


FIGURE 3.2 – Distribution des montants : légitimes vs frauduleuses

Les transactions frauduleuses présentent une distribution fortement asymétrique vers les valeurs élevées, ce qui justifie l'application d'une transformation logarithmique sur le montant (voir section 3.4).

3.2.3 Distribution temporelle de la fraude

La figure 3.3 montre l'évolution du taux de fraude en fonction de l'heure de la journée. Le taux de fraude est significativement plus élevé entre 0h et 5h du matin, ce qui justifie l'inclusion de variables liées à l'heure de la transaction.

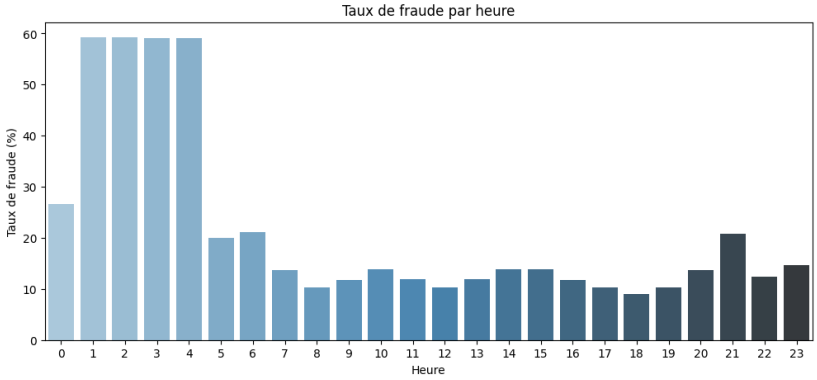


FIGURE 3.3 – Taux de fraude par heure de la journée

3.2.4 Corrélation avec la variable cible

L'analyse de corrélation entre les variables explicatives et la variable cible indiquant la fraude montre que les variables les plus associées à la fraude sont, par ordre d'importance, la distance par rapport au domicile du client, le taux de fraude historique

du pays et de la devise, le nombre de transactions observées dans le pays, ainsi que le montant de la transaction. Ces résultats indiquent que la distance géographique et les indicateurs de risque liés au pays et à la devise constituent les facteurs les plus fortement associés aux transactions frauduleuses dans ce dataset.

Le tableau 3.2 présente les dix variables présentant les corrélations les plus élevées avec `is_fraud`.

TABLEAU 3.2 – Top 10 des variables les plus corrélées avec la cible

Variable	**Corrélation**
distance_from_home	0,635
country_fraud_rate	0,371
currency_fraud_rate	0,371
country_tx_count	0,117
amount_log	0,079
city_size_medium	0,052
amount_vs_customer_avg	0,011
velocity_max_single_amount	0,004
transaction_day	0,003
velocity_num_transactions	0,003

3.3 Prétraitement et nettoyage des données

3.3.1 Audit initial et suppression des doublons

Avant toute analyse, le dataset brut a été audité afin d'identifier les valeurs manquantes, les doublons et les éventuelles anomalies de structure. Seuls les doublons strictement identiques sur l'ensemble des colonnes ont été supprimés, afin d'éviter de retirer par erreur des transactions potentiellement frauduleuses partageant des caractéristiques similaires sans être de véritables doublons techniques.

3.3.2 Traitement du risque de fuite de données

Certaines colonnes du dataset brut présentaient un risque de fuite de données (data leakage), c'est-à-dire une corrélation artificielle avec la variable cible qui n'aurait pas de sens en conditions réelles de production. Ces colonnes ont été supprimées avant toute transformation, comme détaillé dans le tableau 3.3.

Tableau 3.3 – Colonnes supprimées pour éviter le data leakage

Colonne	Justification
Identifiant de transaction	Certains identifiants présentaient un taux de fraude de 100 %, le modèle aurait mémorisé les identifiants frauduleux plutôt que d'apprendre des patterns généralisables.
Numéro de carte	Même problème que l'identifiant de transaction ; en production, un nouveau numéro de carte serait inconnu du modèle.
Empreinte de l'appareil	Forte corrélation artificielle avec la fraude, propre au générateur de données synthétiques.
Adresse IP	Artefact du générateur de données ; en production, une même adresse IP peut correspondre à des comportements à la fois légitimes et frauduleux.
Type d'appareil utilisé	Relation inversée par rapport à la réalité bancaire : la présence physique de la carte est en pratique un facteur de sécurité, pas de risque.
Canal de transaction	Corrélation artificielle inversée par rapport à la réalité, propre au dataset synthétique.

3.3.3 Tri chronologique des transactions

Le tri chronologique des transactions, à partir de leur horodatage, constitue une étape essentielle du pipeline. Sans ce tri, un découpage ultérieur entre jeux d'entraînement et de test pourrait mélanger des transactions passées et futures, ce qui introduirait une fuite d'information temporelle : le modèle pourrait alors apprendre en ayant indirectement accès à des informations issues du futur.

3.4 Ingénierie des caractéristiques

L'ingénierie des caractéristiques constitue l'étape la plus déterminante du pipeline de données. Au total, 52 variables explicatives ont été construites, réparties en plusieurs grandes catégories, chacune répondant à une hypothèse métier précise sur les comportements frauduleux observés dans le secteur financier.

3.4.1 Variables temporelles

Plusieurs variables sont extraites de l'horodatage de chaque transaction : l'heure et le jour de la semaine de la transaction, ainsi qu'un indicateur de week-end. Ces variables permettent de capturer les variations du comportement frauduleux selon le moment de la transaction, mises en évidence lors de l'analyse exploratoire.

3.4.2 Transformation du montant

La distribution des montants étant fortement asymétrique, une transformation logarithmique est appliquée afin de réduire cette asymétrie tout en conservant l'information

portée par les montants élevés, souvent associés aux transactions frauduleuses :

$$\text{montant_log} = \ln(1 + \text{montant}) \tag{3.1}$$

3.4.3 Variables de vélocité

Les informations de vélocité décrivent l'activité récente d'un client : nombre de transactions effectuées dans la dernière heure, montant total associé, nombre de marchands et de pays distincts impliqués et montant maximal observé. Ces variables permettent de détecter des comportements transactionnels anormalement intenses sur une courte période.

3.4.4 Variables cumulatives et historiques client

Afin de prendre en compte l'historique des transactions, plusieurs variables cumulatives ont été construites avant le découpage des données : le nombre de transactions déjà effectuées par un client, le nombre de transactions observées dans un pays donné, ainsi que le montant moyen historique des transactions d'un client. Cette dernière information est utilisée pour comparer le montant de la transaction courante au comportement habituel du client, une différence importante pouvant constituer un indicateur de comportement anormal.

Le tableau 3.4 récapitule les grandes catégories de variables ayant permis de constituer les 52 caractéristiques finales du modèle.

Tableau 3.4 – Récapitulatif des catégories de caractéristiques du modèle

Catégorie	**Description**
Géographique et distance	Présence physique de la carte, distance par rapport au domicile
Temporelles	Heure, jour, jour de semaine, indicateur de week-end
Montant	Montant transformé (échelle logarithmique), comparaison au montant moyen habituel du client
Vélocité	Nombre et montant des transactions récentes, marchands et pays distincts
Agrégats client	Nombre de transactions cumulées du client et du pays
Taux de fraude historiques	Risque marchand, taux de fraude par pays, devise et marchand
Variables catégorielles encodées	Type de carte, taille de ville, catégorie et type de marchand

3.5 Découpage temporel et encodage des variables

3.5.1 Découpage temporel anti-fuite

Afin d'éviter toute contamination des jeux d'entraînement et de validation par des informations futures, le découpage des données en trois ensembles entraînement (70%), validation (15%) et test (15%) a été réalisé sur la base de quantiles temporels, avant tout encodage dépendant de l'étiquette. Le tableau 3.5 présente la répartition obtenue après ce découpage.

Tableau 3.5 – Répartition des données après le découpage temporel

Jeu de données	**Nombre de transactions**	**Taux de fraude**
Entraînement (70%)	5 238 636	20,1%
Validation (15%)	1 122 565	20,6%
Test (15%)	1 122 565	18,9%

3.5.2 Encodage des variables catégorielles

Le taux de fraude historique associé au pays, à la devise et au marchand est calculé exclusivement sur le jeu d'entraînement, par une méthode d'encodage cible (target encoding), puis simplement appliqué aux jeux de validation et de test. Les variables catégorielles à faible cardinalité, telles que le type de carte ou la catégorie de marchand, sont quant à elles encodées par encodage disjonctif complet (one-hot encoding), également ajusté uniquement sur l'ensemble d'entraînement. Cette méthodologie garantit qu'aucune information issue des jeux de validation ou de test n'est utilisée lors de la construction des variables ou de l'entraînement du modèle.

3.6 Modélisation

3.6.1 Rééquilibrage par SMOTE

Afin d'évaluer l'apport du rééquilibrage, deux approches ont été comparées : un entraînement sans rééquilibrage et un entraînement avec SMOTE appliqué sur le jeu d'entraînement après sélection des features. Les résultats sur le jeu de validation montrent que SMOTE améliore significativement toutes les métriques pour XGBoost : AUPRC de 0.900 à 0.963 (+6,3 points), F1 de 0.810 à 0.891 (+8,1 points), et coût métier réduit de 412619 à 310117 (25 %). Ces gains s'expliquent par la meilleure couverture de la frontière de décision autour de la classe minoritaire, qui représente malgré un taux de 20 % une population structurellement différente en termes de distribution de features (montants élevés, distances atypiques, heures nocturnes). SMOTE est donc appliqué uniquement sur le jeu d'entraînement après sélection des 20 features, garantissant l'absence de contamination des jeux de validation et de test.

La figure 3.4 compare les performances du modèle XGBoost avec et sans rééquilibrage.

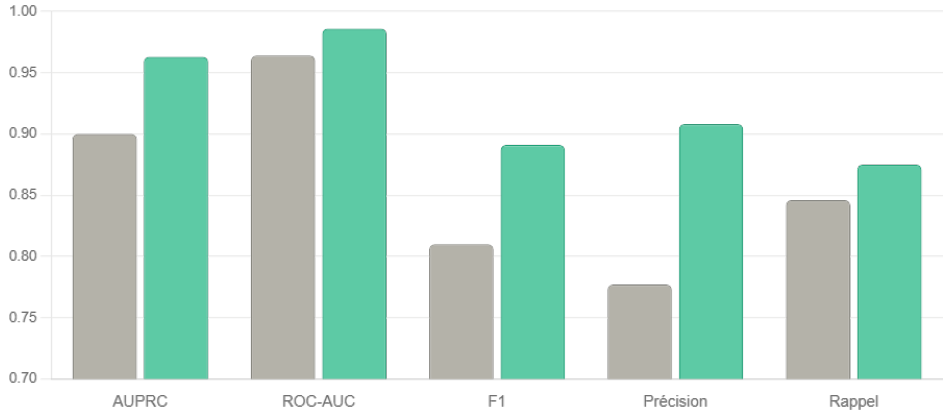


FIGURE 3.4 – Comparaison des métriques de validation avec et sans SMOTE

3.6.2 Sélection des features

Un `RandomForestClassifier` est utilisé comme filtre préliminaire afin de conserver les **20 features les plus discriminantes**, éliminant ainsi le bruit et réduisant le temps d'entraînement. L'importance des features est calculée sur l'ensemble d'entraînement, et les 20 features de plus haute importance sont retenues pour tous les modèles.

3.6.3 Comparaison des modèles candidats

Quatre modèles sont entraînés dans les mêmes conditions afin de sélectionner objectivement le meilleur candidat. Une fonction d'évaluation unifiée `evaluate_model` calcule l'ensemble des métriques à partir des probabilités prédites, avec détermination automatique du seuil optimal par maximisation du F1 Score sur la courbe Precision-Recall.

Modèle 1 : Régression Logistique (Baseline)

La Régression Logistique [10] sert de point de référence interprétable. Elle est entraînée sur les données SMOTE avec standardisation (`StandardScaler`). Ce modèle linéaire simple permet de mesurer le gain apporté par les modèles plus complexes.

Tableau 3.6 – Résultats de la Régression Logistique sur la validation (seuil F1 = 0,6844)

Métrique	**Valeur**
ROC AUC	0,9375
AUPRC	0,8123
Precision	0,7487
Recall	0,7749
F1 Score	0,7616
Accuracy	0,9001
Recall @ 1% FPR	0,3372
Coût métier total	580 250

Modèle 2 : LightGBM

LightGBM [11] est un algorithme de gradient boosting optimisé pour la vitesse, basé sur une stratégie de croissance des arbres par feuilles (*leaf-wise*). Il est entraîné avec *early stopping* (50 rounds sans amélioration sur l'AUC de validation).

Tableau 3.7 – Résultats de LightGBM sur la validation (seuil F1 = 0,5771)

Métrique	**Valeur**
ROC AUC	0,9825
AUPRC	0,9514
Precision	0,8879
Recall	0,8652
F1 Score	0,8764
Accuracy	0,9498
Recall @ 1% FPR	0,7659
Coût métier total	336 645

Modèle 3 : XGBoost

XGBoost [12] est entraîné sur le jeu d'entraînement rééquilibré par SMOTE, qui constitue l'unique mécanisme de gestion du déséquilibre des classes. Le paramètre `scale_pos_weight` n'est pas utilisé ici, car son rôle de pondération devient inutile une fois les classes rééquilibrées par SMOTE. Le modèle est entraîné avec *early stopping* et optimisation de l'AUPRC.

Tableau 3.8 – Résultats de XGBoost sur la validation (seuil F1 = 0,6729)

Métrique	**Valeur**
ROC AUC	0,9860
AUPRC	0,9627
Precision	0,9085
Recall	0,8746
F1 Score	0,8912
Accuracy	0,9560
Recall @ 1% FPR	0,8175
Coût métier total	310 117

Modèle 4 : Réseau de Neurones (MLP)

Un perceptron multicouche (MLP) [13] est implémenté avec PyTorch. L'architecture comporte deux couches cachées (128 et 64 neurones) avec `BatchNorm`, `Dropout(0.3)` et *early stopping* (patience = 5 epochs). L'optimiseur Adam est utilisé avec un taux d'apprentissage de 0,001, et la fonction de perte `BCEWithLogitsLoss` gère nativement la classification binaire.

Tableau 3.9 – Résultats du Réseau de Neurones sur la validation (seuil F1 = 0,5626)

Métrique	**Valeur**
ROC AUC	0,9836
AUPRC	0,9567
Precision	0,9028
Recall	0,8635
F1 Score	0,8827
Accuracy	0,9527
Recall @ 1% FPR	0,7946
Coût métier total	337 024

3.7 Synthèse comparative et sélection du modèle

TABLEAU 3.10 – Comparaison des quatre modèles sur le jeu de validation

Modèle	ROC AUC	AUPRC	F1	Recall	Précision	Coût
Logistic Regression	0,9376	0,8123	0,7616	0,7749	0,7487	580 250
LightGBM	0,9825	0,9514	0,8764	0,8652	0,8879	336 645
XGBoost	**0,9860**	**0,9627**	**0,8912**	**0,8746**	**0,9085**	**310 117**
Neural Network	0,9836	0,9567	0,8827	0,8635	0,9028	337 024

3.7.1 Modèle retenu — XGBoost : justification

La sélection du meilleur modèle est effectuée automatiquement via un `evaluate_model` qui centralise les probabilités de validation de chaque modèle et sélectionne celui qui maximise l'AUPRC. Cette métrique est privilégiée pour les problèmes à classes déséquilibrées, car elle reste insensible au déséquilibre, contrairement à l'accuracy. XGBoost est sélectionné pour les raisons suivantes :

- **Meilleures performances globales** : XGBoost obtient le ROC AUC le plus élevé (0,9860), l'AUPRC la plus haute (0,9627) et le F1 Score maximum (0,8912), surpassant les trois autres modèles sur toutes les métriques discriminantes.
- **Coût métier minimal** : avec un coût total de 310 117 (FN×10 + FP×1), XGBoost génère le moins de pertes financières parmi tous les modèles testés, critère primordial en détection de fraude.
- **Gestion efficace du déséquilibre via SMOTE** : le rééquilibrage par SMOTE, appliqué uniquement sur le jeu d'entraînement, a permis d'obtenir de meilleurs résultats pour XGBoost sans recourir au paramètre `scale_pos_weight`.
- **Recall @ 1% FPR supérieur** : XGBoost détecte 81,75% des fraudes tout en maintenant un taux de faux positifs à 1%, contre 76,59% pour LightGBM et 79,46% pour le réseau de neurones.
- **Explicabilité via SHAP** : intégration de SHAP via un explainer générique (`shap.Explainer`) pour générer des explications locales et globales, garantissant la transparence des décisions du modèle.
- **Reproductibilité et déploiement** : modèle sérialisable via `pickle`, sans dépendance lourde comme PyTorch ou TensorFlow.

3.7.2 Optimisation des hyperparamètres par recherche en grille

Afin de justifier le choix des hyperparamètres plutôt que de les fixer arbitrairement, une recherche en grille a été menée sur trois paramètres clés de XGBoost : `max_depth`, `learning_rate` et `subsample`. Pour limiter le temps de calcul, cette recherche a été réalisée sur un sous-échantillon de 200 000 transactions, avec une validation croisée temporelle à 3 plis (`TimeSeriesSplit`), afin de respecter la même contrainte chronologique que le pipeline principal.

Le tableau 3.11 présente les dix meilleures combinaisons obtenues, classées par AUPRC moyenne.

Tableau 3.11 – Meilleures combinaisons d'hyperparamètres (recherche en grille)

Rang	max_depth	learning_rate	subsample	AUPRC
1	6	0,05	0,8	0,9511
2	6	0,05	0,9	0,9510
3	8	0,05	0,8	0,9509
4	8	0,05	0,9	0,9508
5	8	0,05	0,7	0,9506

La combinaison retenue est `max_depth=6`, `learning_rate=0,05` et `subsample=0,8`, avec une AUPRC de 0,9511. L'écart entre la meilleure et la dixième combinaison reste très faible, ce qui montre que le modèle est peu sensible à ces hyperparamètres dans la plage testée. Ces valeurs sont celles utilisées pour entraîner le modèle XGBoost final.

3.7.3 Sélection du seuil optimal de décision

Le modèle XGBoost retourne, pour chaque transaction, une probabilité de fraude comprise entre 0 et 1. Plutôt que d'utiliser une valeur seuil par défaut, le seuil de décision a été déterminé en maximisant le score F1 3.2 sur le jeu de validation :

$$F1 = \frac{2 \times \text{Précision} \times \text{Rappel}}{\text{Précision} + \text{Rappel}} \tag{3.2}$$

Le score de risque produit par le modèle est ensuite interprété selon une politique de décision à quatre niveaux de sévérité, présentée dans le tableau 3.12.

Tableau 3.12 – Règle de décision à quatre niveaux de sévérité

Niveau	Score de fraude	Action recommandée
Faible	inférieur à 0,30	Approuver la transaction
Moyen	entre 0,30 et 0,67	Surveiller la transaction
Élevé	entre 0,67 et 0,90	Révision manuelle requise
Critique	supérieur ou égal à 0,90	Bloquer immédiatement

3.7.4 Évaluation finale sur le jeu de test

Le modèle XGBoost optimisé a été évalué une unique fois sur le jeu de test, jamais utilisé pendant l'entraînement ni l'optimisation des hyperparamètres. Le tableau 3.13 compare les performances obtenues sur les ensembles de validation et de test.

Tableau 3.13 – Résultats finaux du modèle XGBoost sur validation et test

Métrique	Validation	Test
AUPRC	0,9627	0,9577
Aire sous la courbe ROC	0,9860	0,9852
Précision	0,9085	0,91
Rappel	0,8746	0,85
Score F1	0,8912	0,88
Rappel à 1 % de faux positifs	0,8175	0,8104

Ces résultats confirment la stabilité des performances du modèle entre les jeux de validation et de test, ce qui valide sa bonne capacité de généralisation. La figure 3.5 superpose les métriques des deux jeux de données et quantifie les écarts, tous inférieurs à 0,71 point, écartant tout risque de surapprentissage.

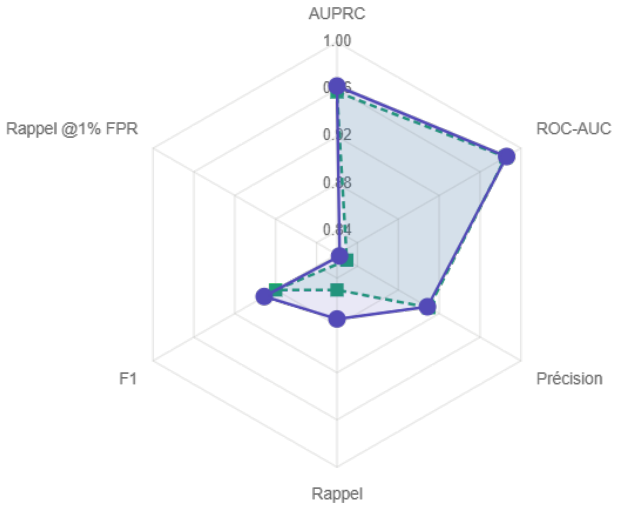


FIGURE 3.5 – Stabilité de généralisation du modèle XGBoost.

3.8 Explicabilité du modèle avec SHAP

L'explicabilité constitue une exigence réglementaire et métier majeure dans le domaine bancaire, notamment au regard du droit à l'explication introduit par l'article 22 du RGPD [2]. Pour y répondre, la méthode SHAP (SHapley Additive exPlanations) [14] a été retenue afin de fournir des explications à deux niveaux : globales, portant sur l'ensemble du dataset, et locales, portant sur une transaction individuelle.

3.8.1 Principe des valeurs de Shapley

SHAP est fondé sur la théorie des jeux coopératifs. Pour une transaction donnée, la valeur de Shapley d'une variable i 3.3 représente sa contribution marginale moyenne à la prédiction, définie comme :

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!\,(|F| - |S| - 1)!}{|F|!} \Big(f(S \cup \{i\}) - f(S) \Big) \tag{3.3}$$

où F désigne l'ensemble des variables, S un sous-ensemble de variables, et $f(S)$ la prédiction du modèle obtenue en n'utilisant que les variables de S. La propriété fondamentale de SHAP est l'additivité, qui relie la prédiction finale $f(x)$ 3.4 à la somme des contributions de chaque variable autour d'une valeur de référence ϕ_0 :

$$f(x) = \phi_0 + \sum_{i=1}^{n} \phi_i \tag{3.4}$$

3.8.2 Importance globale des variables

La figure 3.6 illustre l'importance globale des variables sous la forme d'un graphique en essaim des valeurs SHAP, pour la classe « Fraude ».

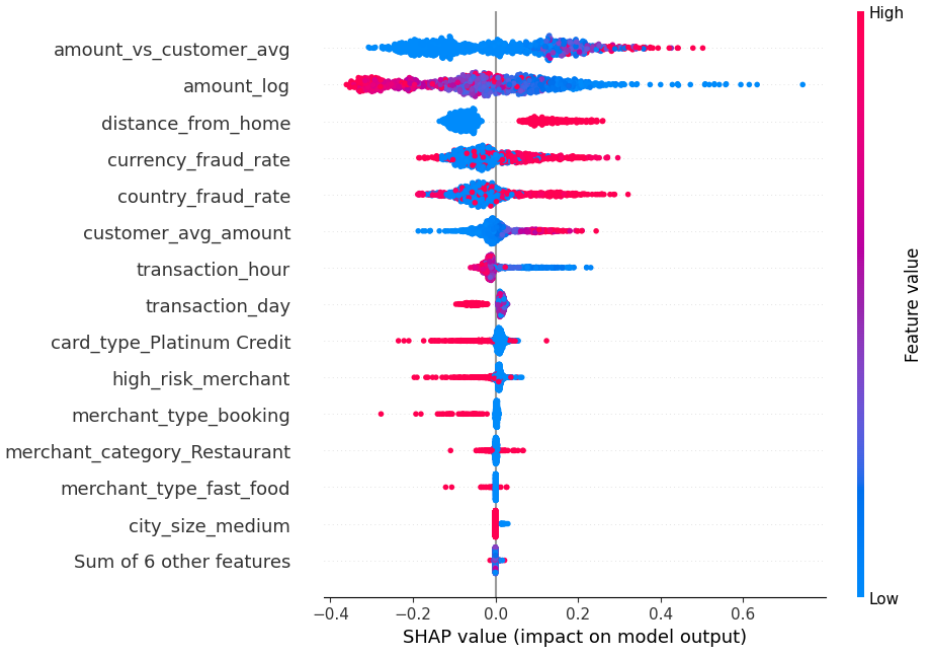


FIGURE 3.6 – Graphique en essaim des valeurs SHAP pour la classe « Fraude »

Le tableau 3.14 présente les sept variables les plus influentes selon cette analyse.

TABLEAU 3.14 – Sept premières variables par importance SHAP globale

Rang	SHAP moyen	Interprétation
1	0,1435	Montant anormalement élevé par rapport à l'historique du client
2	0,1414	Montant de la transaction (échelle logarithmique)
3	0,0951	Transaction géographiquement éloignée du domicile
4	0,0661	Devise associée à un risque historique élevé
5	0,0644	Pays d'origine associé à un risque historique élevé
6	0,0357	Montant moyen habituel du client
7	0,0286	Heure nocturne de la transaction

3.8.3 Explication locale d'une transaction

Au-delà de l'importance globale des variables, chaque transaction individuelle peut être expliquée à l'aide d'un graphique en cascade, illustré sur la figure 3.7.

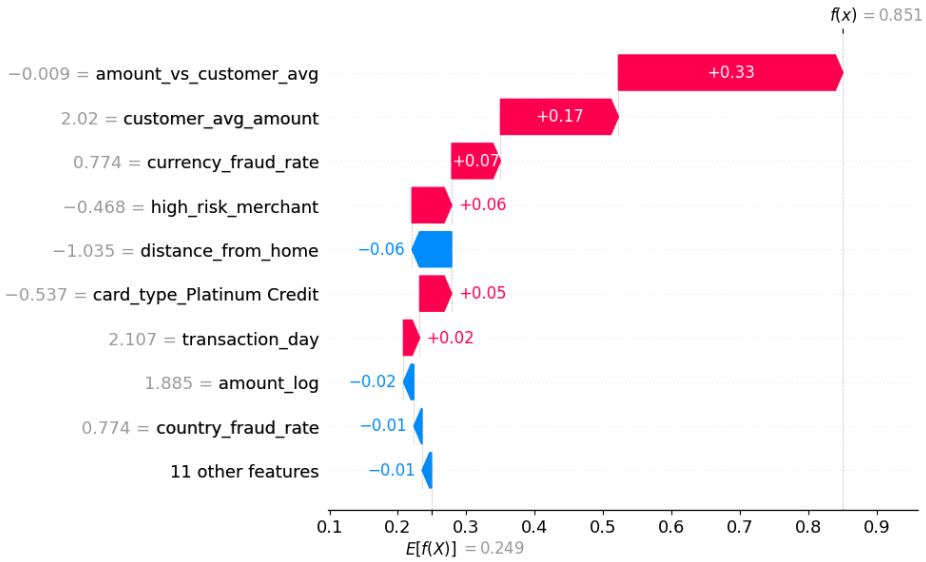


FIGURE 3.7 – Graphique en cascade SHAP : explication d'une transaction frauduleuse

Pour la transaction illustrée, le modèle part d'un score de base proche de 20% (valeur attendue du modèle) pour atteindre un score final supérieur à 99%. Les variables poussant vers la fraude apparaissent en rouge, tandis que celles poussant vers la légitimité apparaissent en bleu, ce qui permet à un analyste de comprendre immédiatement les raisons d'une décision de blocage.

Conclusion

Ce chapitre a présenté l'ensemble du travail réalisé sur les données du projet FraudGuard, depuis la description et l'exploration du dataset jusqu'à la sélection et l'évaluation du modèle XGBoost, en passant par le prétraitement, l'ingénierie des caractéristiques et l'explicabilité des décisions par SHAP. Ces résultats constituent la base du système mis en production, présenté dans le chapitre suivant.

Chapitre 4

API et tableau de bord

Introduction

Ce chapitre présente la mise en production du modèle de détection de fraude, à travers une interface applicative (API) développée avec FastAPI, ainsi que le tableau de bord analytique développé avec React, destiné aux analystes fraude. L'architecture conteneurisée de la plateforme est ensuite détaillée, ainsi que les mécanismes de supervision et de validation mis en place pour garantir sa fiabilité en production.

4.1 Architecture du backend

Le backend du système FraudGuard est organisé de manière modulaire. Au démarrage de l'application, le modèle XGBoost ainsi que l'explainer SHAP associé sont chargés en mémoire, afin de garantir des temps de réponse minimaux lors du traitement des requêtes.

L'application s'articule autour de plusieurs modules fonctionnels distincts :

— Prédiction du score de risque en temps réel,

— Explication des décisions via SHAP,

— Consultation des transactions et des clients,

— Production de statistiques et gestion des alertes,

— Agent conversationnel (Groq),

— Simulateur de transactions.

La figure 4.1 synthétise cette architecture et illustre les interactions entre ces différents composants.

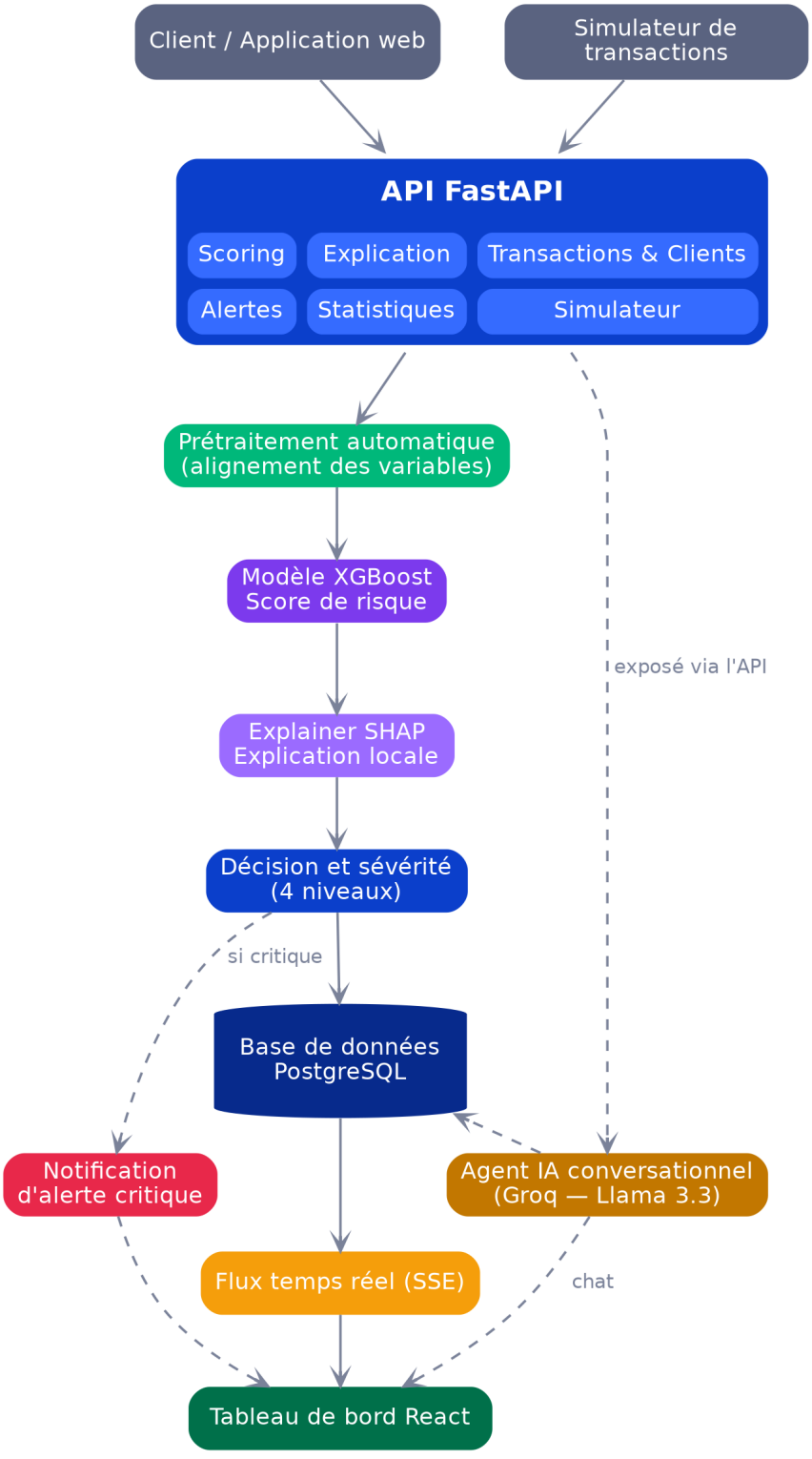


FIGURE 4.1 – Architecture du backend FastAPI de FraudGuard

Le flux de traitement des transactions suit les étapes suivantes :

— Les transactions, qu'elles proviennent d'un client réel ou du simulateur, sont reçues par l'API FastAPI puis prétraitées.

— Le modèle XGBoost calcule un score de risque pour chaque transaction.

— L'explainer SHAP produit l'explication locale (Waterfall) associée à la décision.

— La décision finale, accompagnée de son explication, est persistée dans la base de données PostgreSQL.

— La transaction est diffusée en temps réel vers le tableau de bord via un flux SSE (Server-Sent Events).

— Une notification est déclenchée lorsque la sévérité de la transaction est critique.

— L'agent conversationnel s'appuie sur l'API et sur l'historique des transactions pour répondre aux requêtes formulées en langage naturel.

4.2 Fonctionnalités exposées par l'API

4.2.1 Prédiction du score de risque en temps réel

La fonctionnalité centrale du système consiste à recevoir les caractéristiques d'une transaction, à les aligner automatiquement avec celles attendues par le modèle, puis à calculer un score de risque compris entre 0 et 1. Ce score est ensuite converti en décision opérationnelle (approbation, surveillance, révision manuelle ou blocage) selon le seuil et la règle de sévérité présentés au chapitre précédent. L'explication SHAP associée est calculée dans le même traitement, puis l'ensemble (score, décision, sévérité et explication) est persisté en base de données à des fins d'audit et de traçabilité. Les mesures réalisées montrent une latence de traitement de l'ordre de quelques millisecondes par transaction, compatible avec un usage en production bancaire.

Le tableau 4.1 présente, à titre indicatif, quelques-uns des champs caractérisant une transaction soumise au système.

Tableau 4.1 – Exemples de champs caractérisant une transaction

Champ	**Description**
Identifiant de transaction	Référence unique de la transaction
Identifiant client (optionnel)	Référence du client associé à la transaction
Distance par rapport au domicile	Distance géographique entre le lieu de la transaction et le domicile habituel du client
Heure et jour de la transaction	Moment auquel la transaction a été effectuée
Montant (échelle logarithmique)	Montant de la transaction, transformé pour réduire son asymétrie
Indicateurs de vélocité	Activité récente du client (nombre et montant des transactions, marchands et pays distincts)
Taux de fraude historiques	Niveau de risque associé au pays, à la devise et au marchand

4.2.2 Explication des décisions

Une fonctionnalité dédiée permet d'obtenir, pour une transaction donnée, le détail de son explication SHAP : la valeur de référence du modèle, ainsi que les principales variables ayant contribué à la décision, avec leur contribution respective.

4.2.3 Simulateur de transactions

Pour faciliter les démonstrations et les tests de charge, le système intègre un simulateur capable de générer un flux continu de transactions, avec un taux de fraude paramétrable. Ce simulateur maintient un état par client, ce qui lui permet de générer des transactions cohérentes avec l'historique simulé de chaque client, et peut être démarré, arrêté ou interrogé sur son état à tout moment.

4.2.4 Diffusion en temps réel

Les transactions scorées, qu'elles proviennent du simulateur ou d'un flux réel, sont diffusées en temps réel vers le tableau de bord à l'aide d'un mécanisme de flux d'événements, permettant une mise à jour continue de l'interface sans nécessiter de rafraîchissement manuel.

4.2.5 Agent conversationnel

Le système intègre un agent conversationnel, fondé sur un grand modèle de langage (Llama 3.3 70B, exploité via la plateforme Groq), permettant à un analyste d'interroger le système en langage naturel. L'agent ne se contente pas d'interroger le LLM : il orchestre une chaîne de traitement en trois couches qui garantit des réponses précises, contextualisées et traçables.

Couche 1 – Détection d'intention par règles
Un module de détection d'intention, basé sur des expressions régulières et des mots-clés, classe chaque message utilisateur en sept catégories. Cette classification détermine quelle fonction d'investigation sera appelée, évitant ainsi de transmettre au LLM des données non pertinentes.

Couche 2 – Fonctions d'investigation spécialisées
Chaque intention déclenche une fonction d'investigation dédiée, qui interroge la base de données PostgreSQL pour extraire le contexte métier adapté. Par exemple, la fonction SHAP récupère le `shap_json` d'une transaction précise et construit un contexte textuel structuré (score, décision, expected value, top features). La fonction tendance agrège les transactions des dernières 24 heures par heure. La fonction client extrait automatiquement l'identifiant depuis le message naturel (par exemple `CUST_01234`), puis retourne la fiche détaillée du client avec son historique et ses statistiques.

Couche 3 – Génération de la réponse par le LLM

Le modèle Llama 3.3 70B, accessible via l'API Groq, reçoit le contexte structuré ainsi qu'un prompt système strict. Ce prompt impose de répondre en français, de

se limiter à huit phrases maximum et de conclure par une recommandation d'action concrète. L'historique conversationnel est transmis à chaque appel pour permettre des échanges multi-tours cohérents. La température du modèle est fixée à 0,2, favorisant des réponses factuelles et déterministes, ce qui garantit la fiabilité des explications fournies aux analystes.

La figure 4.2 schématise l'ensemble du processus.

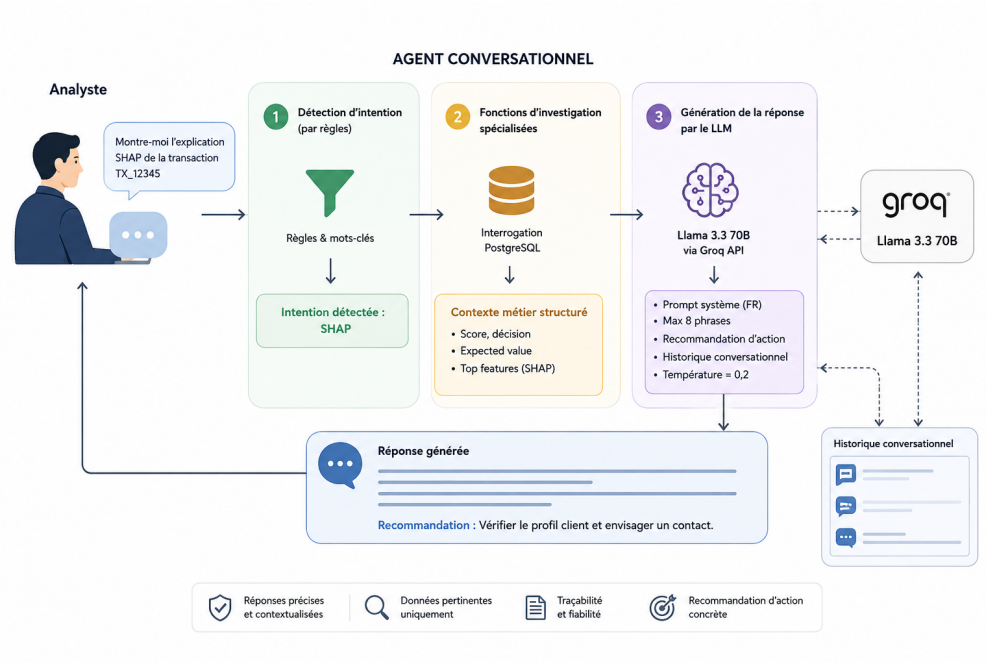


FIGURE 4.2 – Architecture de l'agent conversationnel FraudGuard.

4.3 Tableau de bord React

4.3.1 Choix technologiques

Le tableau de bord a été développé avec React, en s'appuyant sur la bibliothèque de composants Material UI pour l'interface, sur Recharts pour les visualisations graphiques, et sur un mécanisme de flux d'événements pour la réception des données en temps réel. Le tableau 4.2 résume les choix technologiques retenus pour le frontend.

TABLEAU 4.2 – Stack technologique du frontend FraudGuard

Technologie	**Rôle**
React	Bibliothèque de construction des composants d'interface
Material UI	Système de conception et composants visuels
React Router	Navigation au sein de l'application
Recharts	Graphiques (courbes, barres, secteurs, radar)
Axios	Appels vers l'API
Flux d'événements	Connexion temps réel avec le backend

4.3.2 Identité visuelle

Une identité visuelle propre a été définie pour le tableau de bord, combinant une palette de couleurs cohérente : bleu pour les éléments principaux, vert pour les indicateurs positifs, rouge, orange et violet pour les niveaux de sévérité et d'alerte avec une typographie moderne associant une police de titre marquée et une police de texte sobre. Les composants visuels (cartes, boutons, menu latéral) suivent des règles de style homogènes, garantissant une expérience utilisateur cohérente sur l'ensemble des pages.

4.3.3 Présentation des pages du tableau de bord

Le tableau de bord se compose de neuf pages, organisées dans le menu latéral en trois groupes : un groupe principal (tableau de bord, transactions, clients, alertes, investigation), un groupe modèle (performance), et un groupe système (agent IA, pipeline de test, paramètres).

La page principale, décrite ci-dessous, centralise les indicateurs clés et les outils d'analyse rapide du système.

— **En-tête** : rappelle le modèle actif (XGBoost), le seuil de décision (0,673), et donne un accès direct au démarrage de la simulation ainsi qu'aux notifications.

— **Indicateurs clés (KPIs)** : affiche en temps réel le volume total de transactions, le nombre de fraudes détectées (blocage ou révision manuelle), le taux de fraude, la latence moyenne de scoring, et le score de performance du modèle (AUPRC) comparé à une valeur de référence.

— **Panneaux d'analyse** : propose une courbe de tendance des transactions légitimes et frauduleuses, un diagramme circulaire de la répartition par sévérité, ainsi qu'un récapitulatif des métriques de performance du modèle (AUPRC, aire sous la courbe ROC, seuil de décision et latence p95).

— **Accès rapide** : offre un aperçu de flux en temps réel des transactions, un panneau d'explication SHAP mettant en avant la dernière transaction frauduleuse détectée, ainsi qu'un accès direct à l'agent conversationnel FraudGuard avec des suggestions de questions fréquentes.

La figure 4.3 illustre la page principale du tableau de bord, qui centralise ces différents composants.

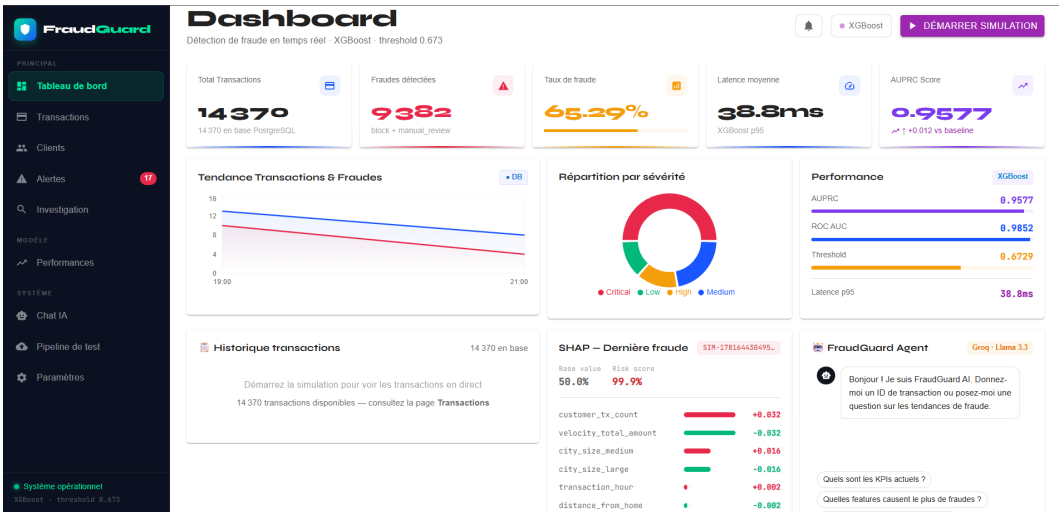


FIGURE 4.3 – Page Dashboard : vue principale avec indicateurs temps réel

Page Pipeline de test

La page Pipeline de test permet d'importer un nouveau jeu de données et d'évaluer le modèle sur ces données. Pour cela, un pipeline de prétraitement automatique a été mis en place. Il assure successivement :

— la lecture du fichier importé avec détection automatique du format,

— l'identification des types de colonnes et des valeurs manquantes,

— le nettoyage des données,

— la reconstruction des variables d'ingénierie,

— l'alignement avec les variables attendues par le modèle,

— le calcul des scores par lot,

— le calcul des métriques de performance et leur sauvegarde.

La figure 4.4 illustre cette page.

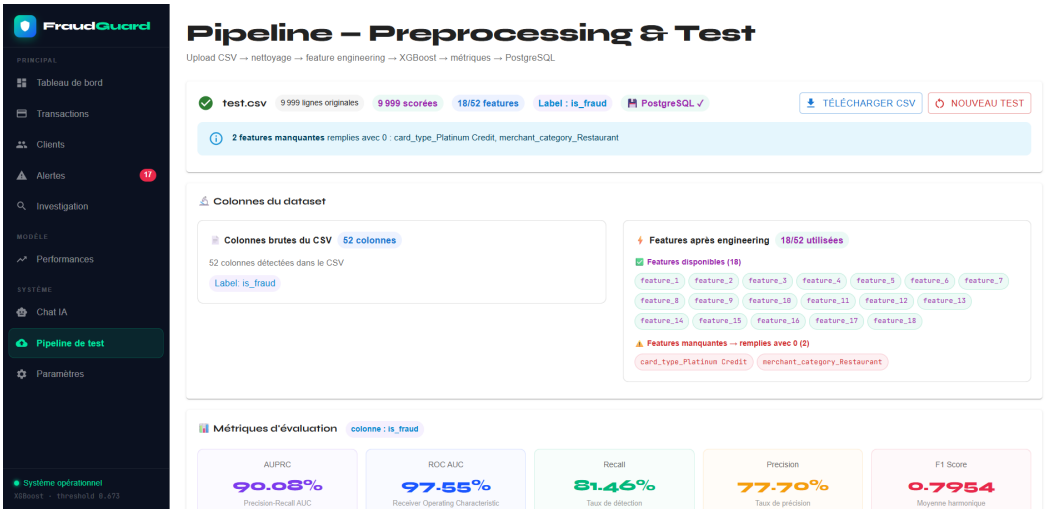


FIGURE 4.4 – Page Pipeline de test : import de données et résultats

Page Transactions

La figure 4.5 affiche l'historique complet des transactions scorées, avec des possibilités de filtrage et un accès direct au rapport d'investigation (score et explication SHAP) de chaque ligne.

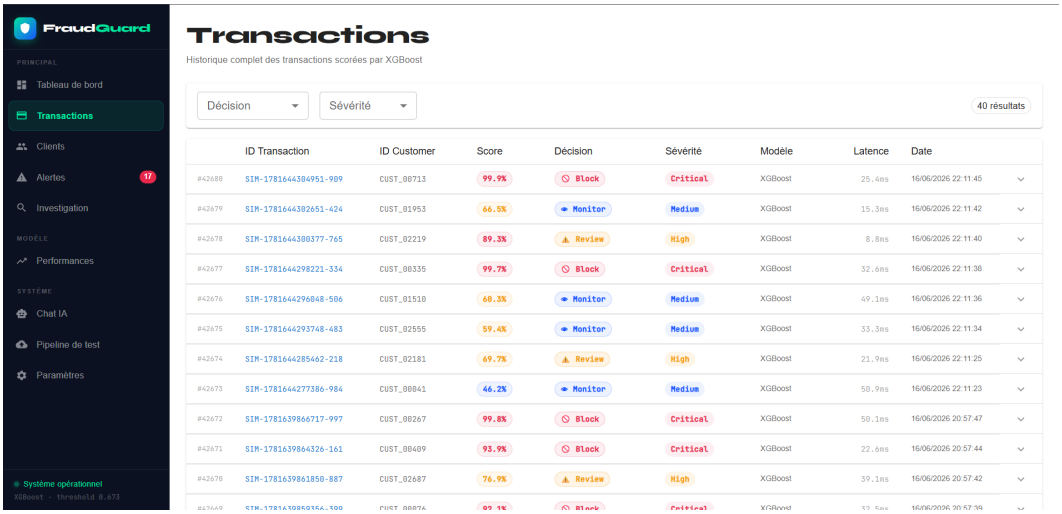


FIGURE 4.5 – Page Transactions

Page Clients

La figure 4.6 présente la liste des clients avec leurs indicateurs d'activité transactionnelle (nombre de transactions, répartition par décision, sévérité et taux de fraude).

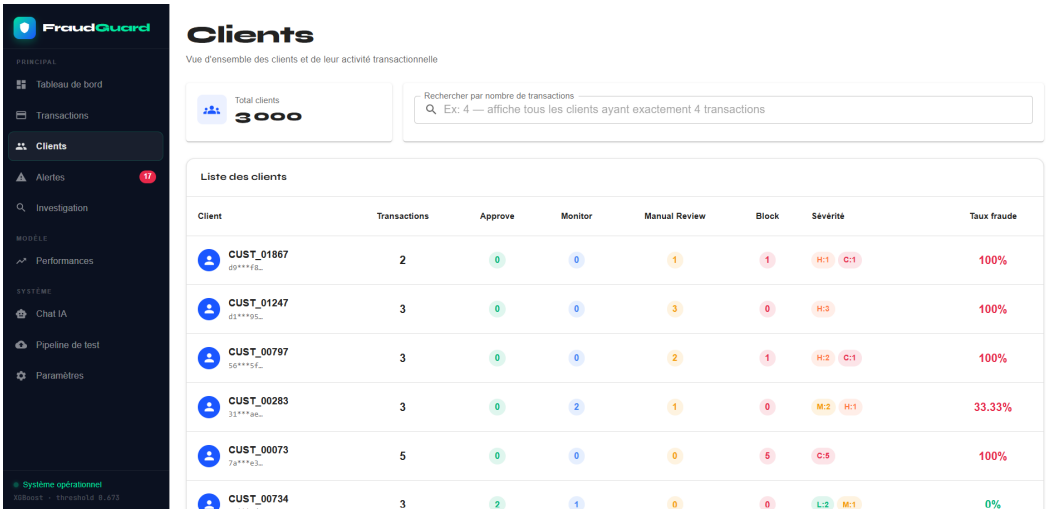


FIGURE 4.6 – Page Clients

Page Alertes

La figure 4.7 regroupe les transactions jugées les plus risquées et permet d'accéder rapidement à leur rapport d'investigation.

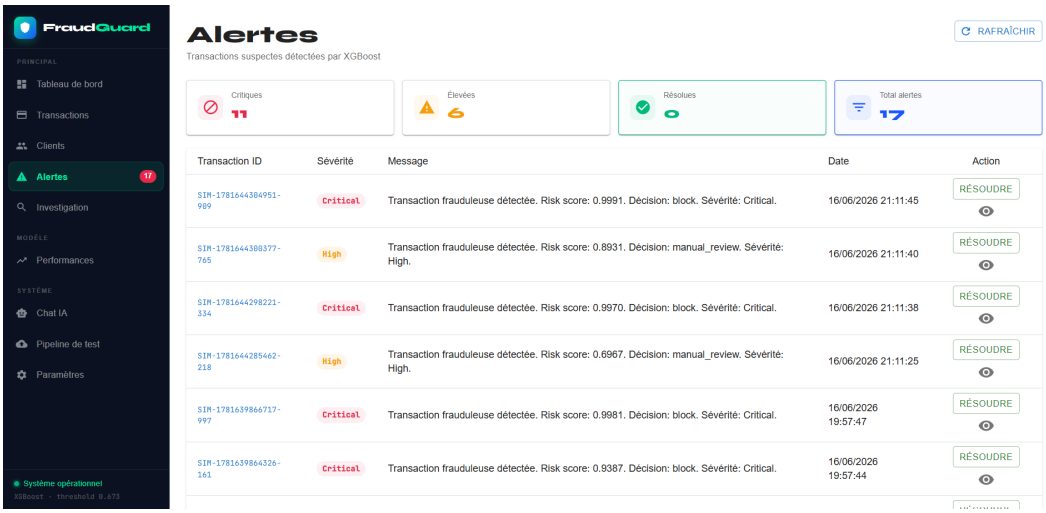


FIGURE 4.7 – Page Alertes

Page Investigation

La figure 4.8 regroupe les fonctionnalités d'analyse approfondie du système. Elle permet d'interroger l'agent conversationnel pour explorer des patterns de fraude, de rechercher une transaction à partir de son identifiant et de générer un rapport explicatif combinant score de risque et graphique SHAP (Waterfall). Ce rapport peut être exporté au format PDF, et l'historique des investigations est conservé dans une table dédiée de la base de données.

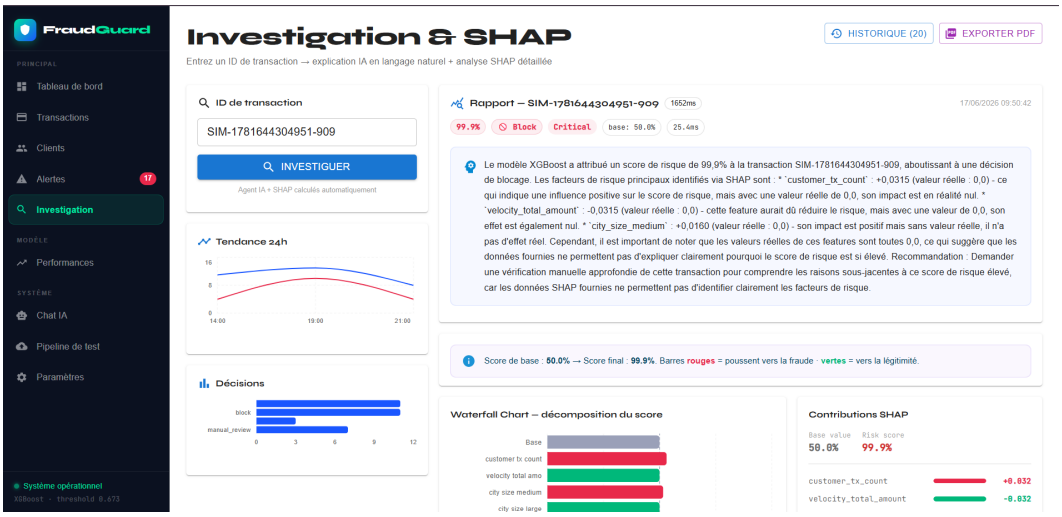


FIGURE 4.8 – Page Investigation

Page Performance

La figure 4.9 présente les métriques de performance du modèle (AUPRC, ROC AUC, F1, précision, rappel) ainsi que la distribution des scores de risque récents.

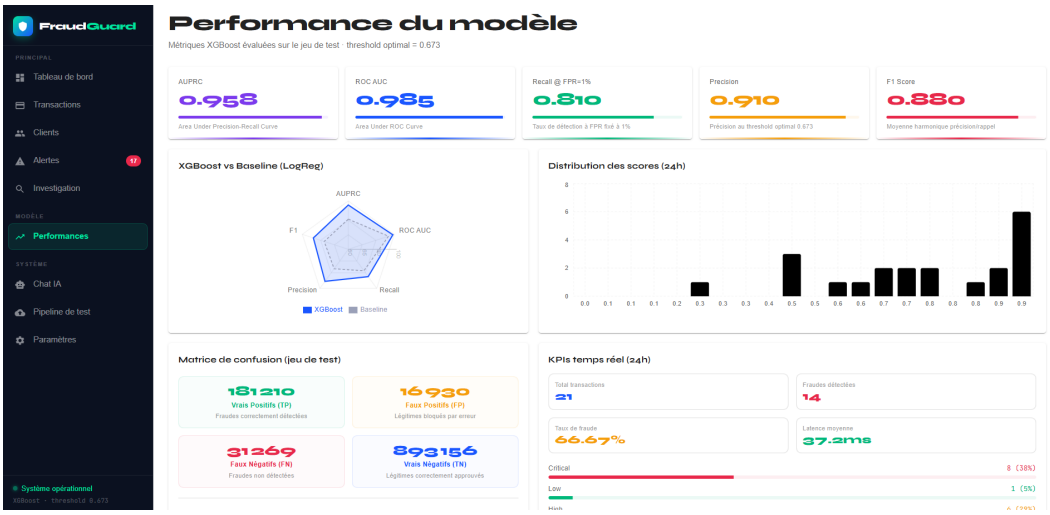


FIGURE 4.9 – Page Performance

Page Agent IA

La figure 4.10 propose une interface de conversation complète avec l'agent IA, permettant d'interroger le système sur les transactions, les tendances de fraude ou les indicateurs globaux.

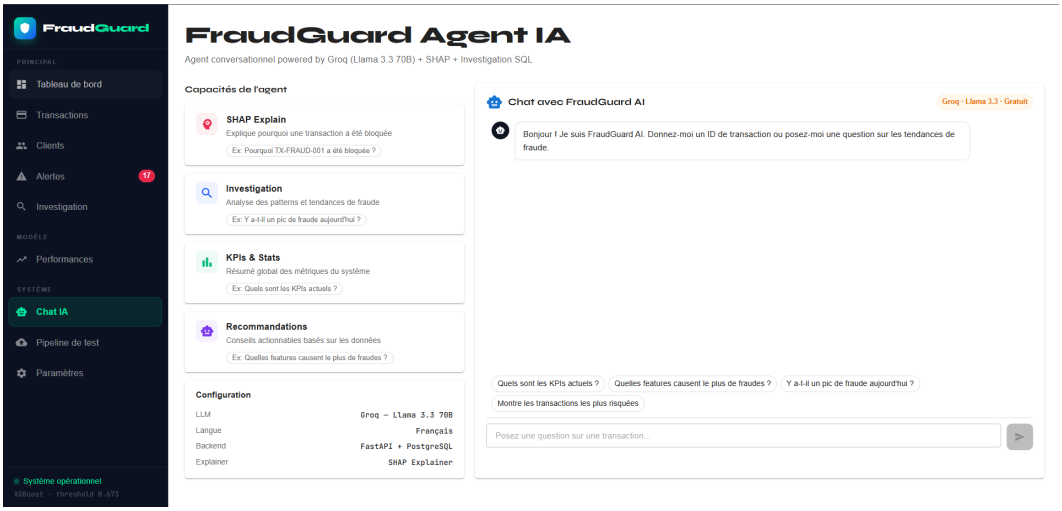


FIGURE 4.10 – Page Agent IA

Exemples des chats

L'agent conversationnel intégré à la plateforme permet à l'analyste fraude d'interroger le système en langage naturel. Les figures 4.11, 4.12, 4.13, 4.14 suivantes illustrent plusieurs scénarios d'utilisation représentatifs.

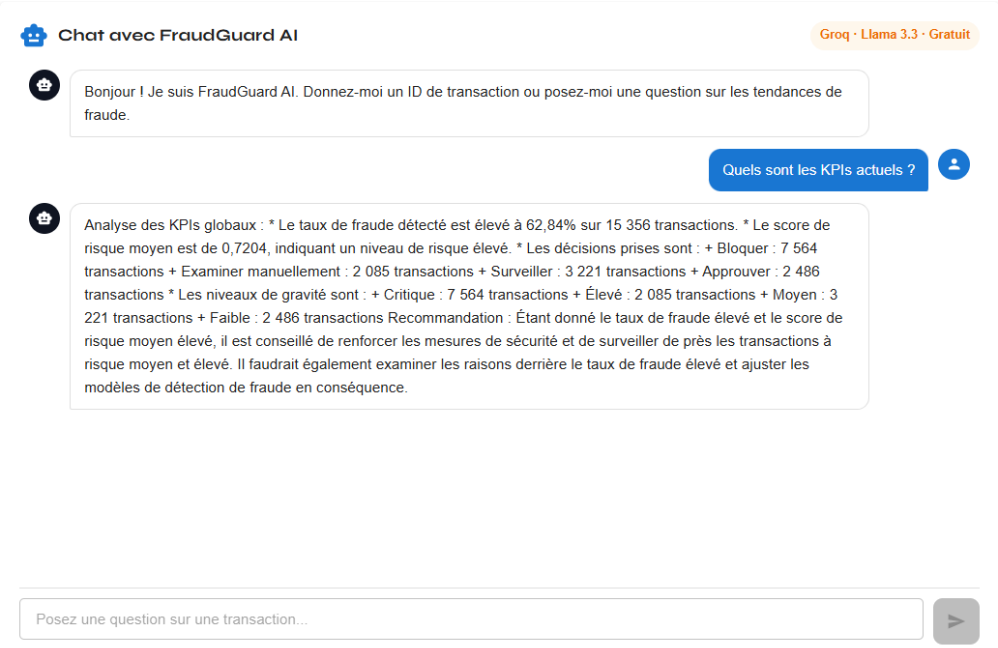


FIGURE 4.11 – Agent IA — consultation des KPIs globaux

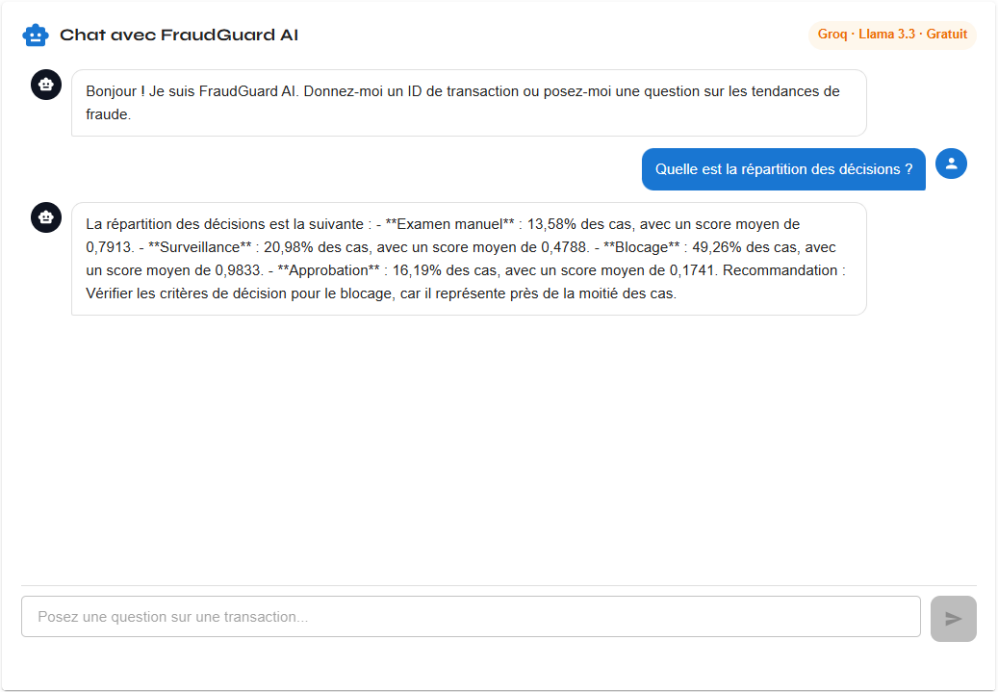


FIGURE 4.12 – Agent IA — Répartition des décisions

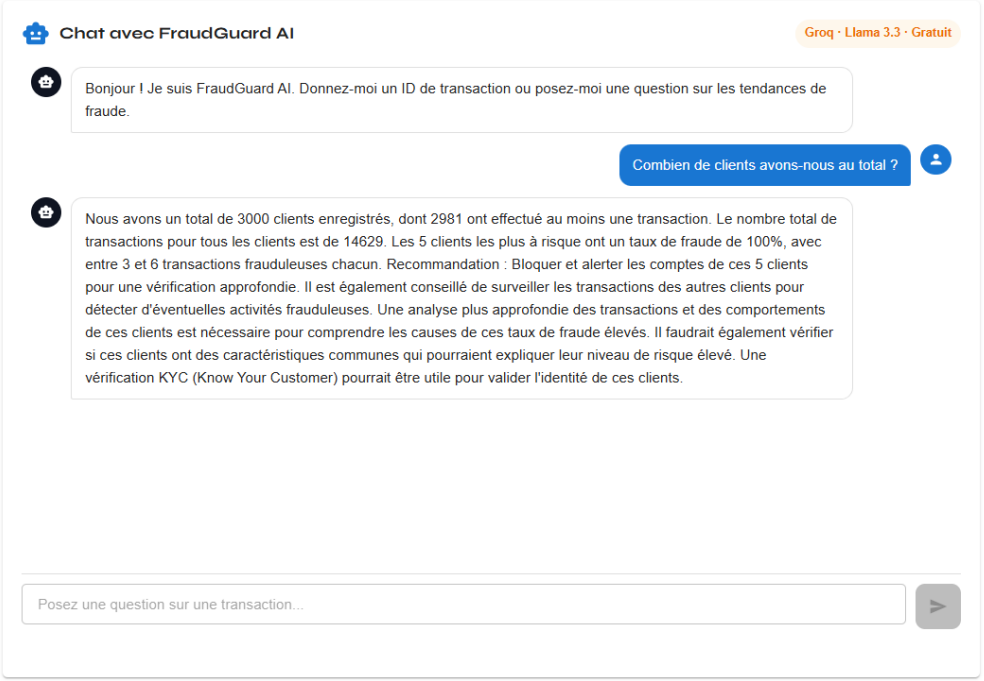


FIGURE 4.13 – Agent IA — Vue générale clients

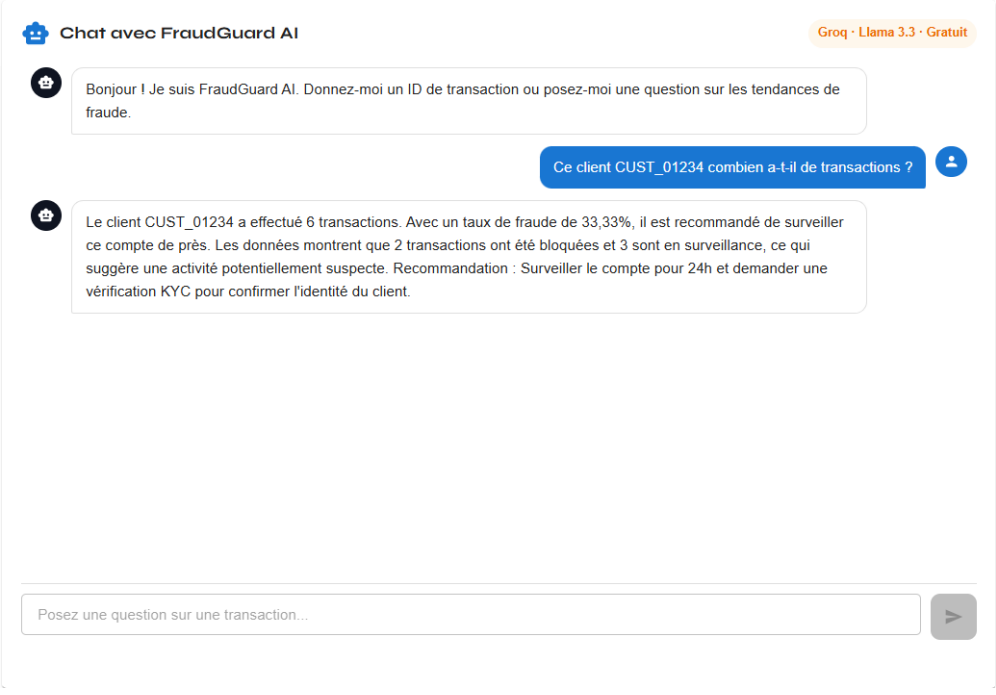


FIGURE 4.14 – Agent IA — investigation d'un profil client

Page Paramètres

La figure 4.15 centralise les réglages de l'application : configuration du seuil de décision, paramètres du simulateur, configuration de l'agent conversationnel et des notifications.

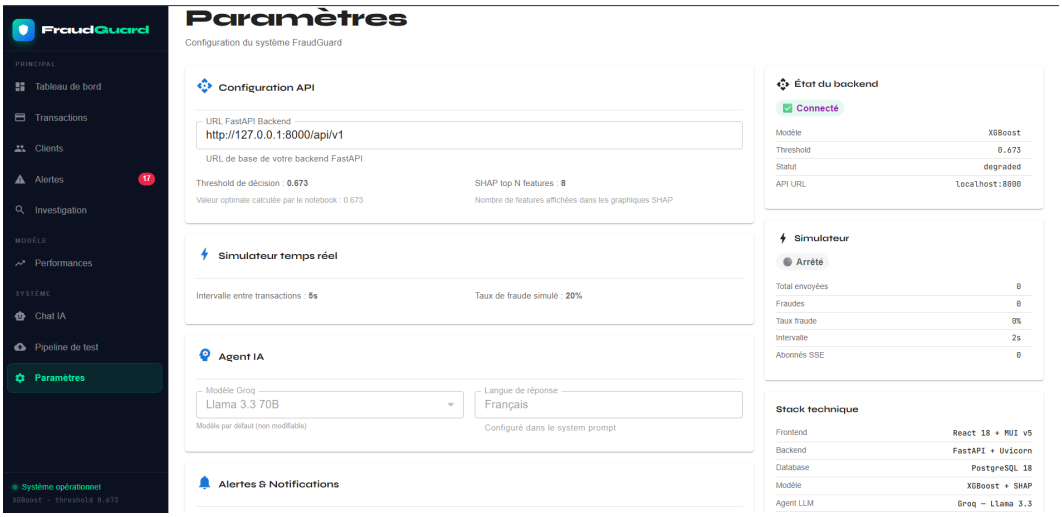


FIGURE 4.15 – Page Paramètres

4.4 Architecture conteneurisée

L'ensemble des composants du système est conteneurisé à l'aide de Docker Compose, ce qui garantit un déploiement homogène et reproductible. Le tableau 4.3 récapitule les neuf conteneurs déployés, leur rôle, et le port éventuellement publié vers la machine hôte.

TABLEAU 4.3 – Conteneurs Docker du système FraudGuard

Conteneur	**Rôle**	**Port publié**
fraud_api	API FastAPI (scoring, explication, agent)	8000
fraudguard-frontend	Tableau de bord React	3000
fraudguard-db	Base de données PostgreSQL	5432
fraudguard-redis	Limitation de débit	aucun
fraudguard-prometheus	Collecte des métriques	9090
fraudguard-grafana	Supervision visuelle	3001
fraudguard-alertmanager	Notification des alertes	9093
fraudguard-node-exporter	Métriques système	9100
fraudguard-postgres-exporter	Métriques de la base de données	9187

Pour garantir la reproductibilité des images, chaque service conteneurisé repose sur un fichier `Dockerfile` dédié. Le backend s'appuie sur une image Python légère (`python:3.11-slim`) pour empaqueter l'API FastAPI, tandis que le frontend adopte une construction en deux étapes (*multi-stage build*) : une première phase avec Node.js pour compiler le bundle statique, et une seconde avec Nginx pour le servir.

La figure 4.16 atteste de la présence de ces fichiers de définition dans les dossiers respectifs du projet.

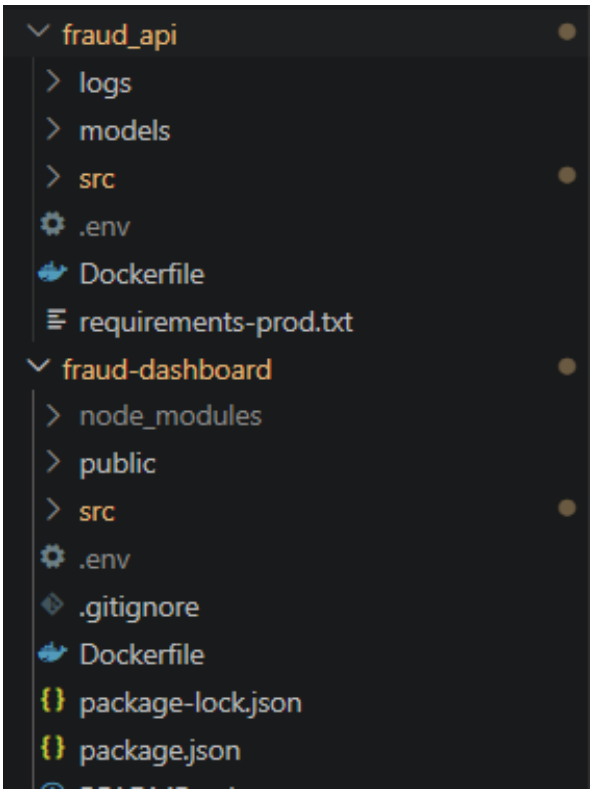


FIGURE 4.16 – Présence des Dockerfiles dans l'arborescence du projet (backend et frontend)

Tous ces conteneurs communiquent à travers un réseau Docker dédié, en se joignant directement par leur nom de service plutôt que par leur adresse IP. Des volumes nommés assurent la persistance des données de PostgreSQL, de Prometheus et de Grafana, un mécanisme de vérification de santé garantit que l'API ne démarre qu'après la base de données, et les secrets (mot de passe, clé de l'agent IA) sont injectés par variables d'environnement plutôt qu'inscrits en dur dans les images. La figure 4.17 illustre cette organisation logique des services et de leurs dépendances.

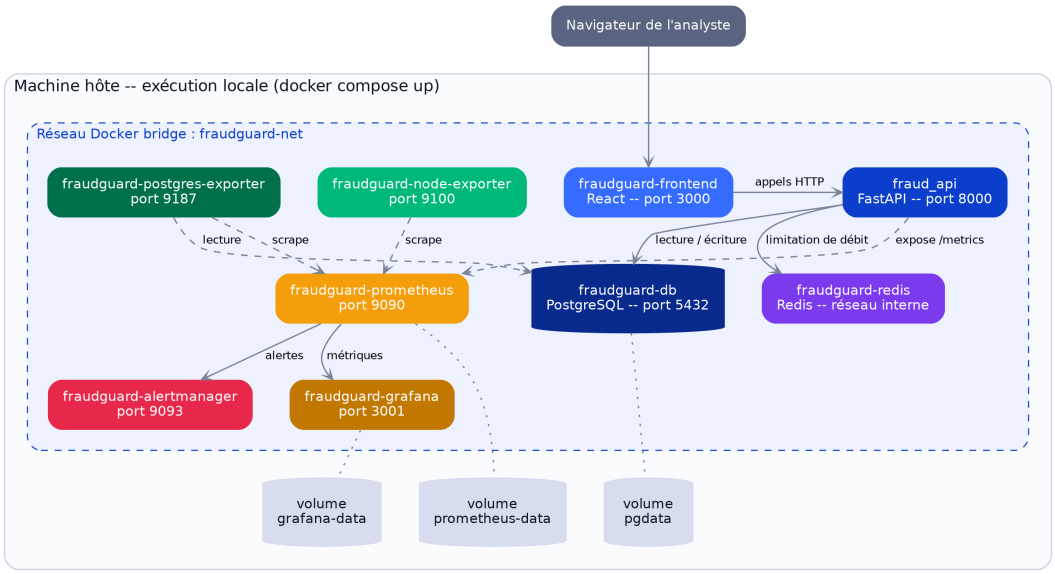


FIGURE 4.17 – Schéma d'architecture des services conteneurisés et de leurs dépendances

4.4.1 Exposition réseau et exécution

Seuls les services devant être joignables depuis l'extérieur du réseau Docker publient un port : le tableau de bord pour les analystes, ainsi que les interfaces de Grafana, Prometheus et AlertManager lorsqu'un accès direct est utile pour l'administration. Redis, à l'inverse, n'a volontairement aucun port publié : il n'est accessible que par les autres conteneurs du réseau interne, ce qui pourrait, dans une optique de durcissement de la sécurité, être étendu à la base de données et aux exporters de métriques.

Dans le cadre de ce projet, cette stack est exécutée localement sur la machine de développement à l'aide de la commande `docker compose up`. La figure 4.18 présente une capture des conteneurs effectivement actifs sur la machine de développement, confirmant le bon démarrage de l'ensemble des services décrits précédemment.



NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
fraud_api	devops-backend	"uvicorn src.api.mai…"	backend	9 seconds ago	Up 6 seconds (health: starting)	0.0.0.0:800
0->8000/tcp, [::]:8000->8000/tcp						
fraudguard-alertmanager	prom/alertmanager:v0.26.0	"/bin/alertmanager -…"	alertmanager	43 hours ago	Up 43 hours	0.0.0.0:909
3->9093/tcp, [::]:9093->9093/tcp						
fraudguard-db	postgres:15-alpine	"docker-entrypoint.s…"	db	2 days ago	Up 2 days (healthy)	0.0.0.0:543
2->5432/tcp, [::]:5432->5432/tcp						
fraudguard-frontend	devops-frontend	"/docker-entrypoint.…"	frontend	7 hours ago	Up 7 hours	0.0.0.0:300
0->80/tcp, [::]:3000->80/tcp						
fraudguard-grafana	grafana/grafana:10.2.0	"/run.sh"	grafana	2 days ago	Up 2 days	0.0.0.0:300
1->3000/tcp, [::]:3001->3000/tcp						
fraudguard-node-exporter	prom/node-exporter:v1.7.0	"/bin/node_exporter …"	node-exporter	2 days ago	Up 2 days	0.0.0.0:910
0->9100/tcp, [::]:9100->9100/tcp						
fraudguard-postgres-exporter	prometheuscommunity/postgres-exporter:v0.15.0	"/bin/postgres_expor…"	postgres-exporter	2 days ago	Up 2 days	0.0.0.0:918
7->9187/tcp, [::]:9187->9187/tcp						
fraudguard-prometheus	prom/prometheus:v2.48.0	"/bin/prometheus --c…"	prometheus	2 days ago	Up 43 hours	0.0.0.0:909
0->9090/tcp, [::]:9090->9090/tcp						
fraudguard-redis	redis:7-alpine	"docker-entrypoint.s…"	redis	2 days ago	Up 2 days	6379/tcp

FIGURE 4.18 – Capture d'écran des conteneurs Docker actifs (VS Code)

4.4.2 Supervision et observabilité

Au-delà des métriques système exposées nativement par les exporters Node Exporter et Postgres Exporter, l'API FastAPI expose ses propres métriques applicatives sur un point de terminaison dédié (`/metrics`), au format attendu par Prometheus. Ce dernier interroge périodiquement (*scraping*) ce point de terminaison, selon un modèle de

collecte par extraction (*pull*), et stocke les séries temporelles obtenues. Les métriques exposées couvrent notamment le volume de transactions traitées, la latence de scoring, le taux d'erreurs, ainsi que la performance courante du modèle (AUPRC, ROC AUC). Grafana se connecte ensuite à Prometheus comme source de données pour construire des tableaux de bord de supervision en temps réel, tandis qu'AlertManager déclenche des notifications lorsque certains seuils sont dépassés.

Surveillance de la dérive du modèle (PSI)

Parmi les métriques applicatives exposées, un indicateur de dérive des données (*data drift*) a été mis en place, fondé sur le *Population Stability Index* (PSI). Ce dernier permet de détecter, sans disposer du label réel de fraude, un éloignement progressif de la distribution des transactions traitées en production par rapport à celle du jeu d'entraînement.

Le PSI compare, pour chaque variable surveillée, la répartition des valeurs courantes à une référence calculée sur le jeu d'entraînement (déciles), selon la formule :

$$PSI = \sum_{i=1}^{10} (\%_{\text{actuel},i} - \%_{\text{attendu},i}) \times \ln\left(\frac{\%_{\text{actuel},i}}{\%_{\text{attendu},i}}\right)$$

avec les seuils d'interprétation usuels : un PSI inférieur à 0,10 indique une distribution stable, une valeur comprise entre 0,10 et 0,25 signale une dérive modérée à surveiller, et un PSI supérieur à 0,25 justifie un ré-entraînement du modèle.

Cette méthode, basée sur un découpage en déciles, n'est pertinente que pour des variables continues, stationnaires dans le temps et effectivement retenues parmi les 20 caractéristiques finales du modèle. Le suivi a donc été restreint aux variables calculées de manière identique et indépendante de l'historique du client, telles que le montant transformé (`amount_log`), la distance par rapport au domicile (`distance_from_home`) et les variables temporelles. Les variables cumulatives (historique client) et les indicateurs de risque encodés par target encoding ont été exclus du suivi, non reproductibles fidèlement par l'environnement de démonstration simulé.

Le calcul est effectué périodiquement par une tâche de fond FastAPI, exposé sous forme de *gauge* Prometheus par variable, puis visualisé dans un tableau de bord Grafana dédié (figure 4.19).

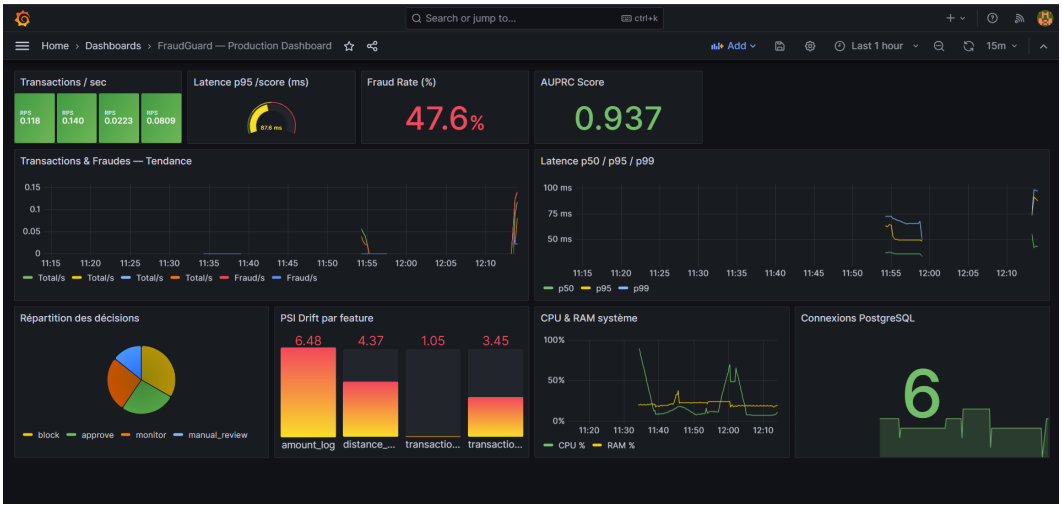
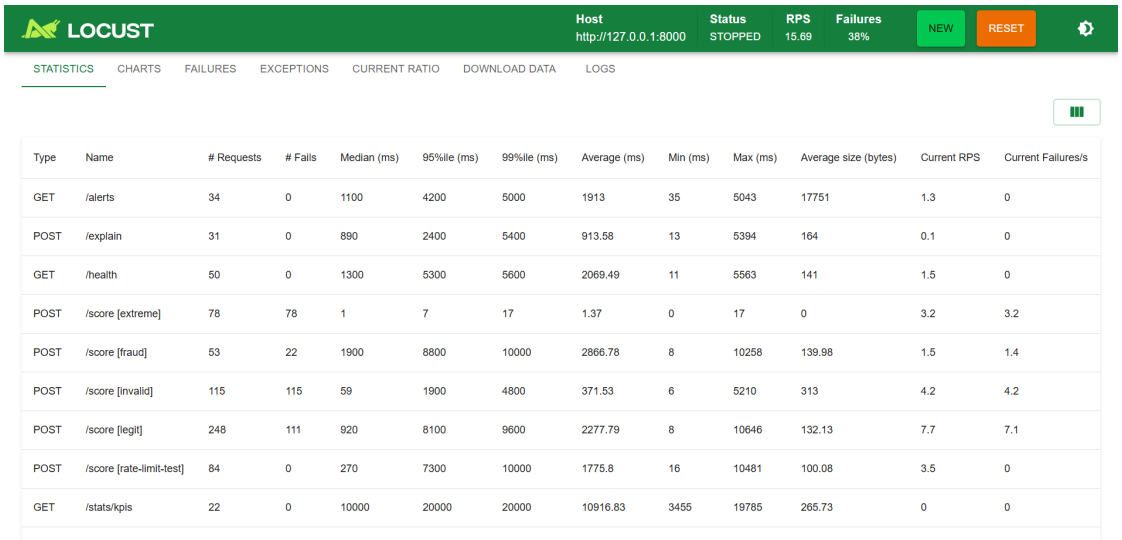


FIGURE 4.19 – Suivi du PSI par variable dans Grafana

4.4.3 Tests de charge

Afin de valider le comportement du système sous forte concurrence, une campagne de tests de charge a été menée à l'aide de Locust, en simulant 100 utilisateurs simultanés répartis en trois profils : un profil analyste consultant le tableau de bord, un profil utilisateur standard soumettant des transactions valides (frauduleuses et légitimes), et un profil attaquant envoyant des requêtes malformées ou en rafale, destiné à valider la robustesse de la validation des entrées et du mécanisme de limitation de débit (BNF3).

La figure 4.20 présente les résultats obtenus après une minute de montée en charge progressive (*ramp-up* de 2 utilisateurs par seconde).



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/alerts	34	0	1100	4200	5000	1913	35	5043	17751	1.3	0
POST	/explain	31	0	890	2400	5400	913.58	13	5394	164	0.1	0
GET	/health	50	0	1300	5300	5600	2069.49	11	5563	141	1.5	0
POST	/score [extreme]	78	78	1	7	17	1.37	0	17	0	3.2	3.2
POST	/score [fraud]	53	22	1900	8800	10000	2866.78	8	10258	139.98	1.5	1.4
POST	/score [invalid]	115	115	59	1900	4800	371.53	6	5210	313	4.2	4.2
POST	/score [legit]	248	111	920	8100	9600	2277.79	8	10646	132.13	7.7	7.1
POST	/score [rate-limit-test]	84	0	270	7300	10000	1775.8	16	10481	100.08	3.5	0
GET	/stats/kpis	22	0	10000	20000	20000	10916.83	3455	19785	265.73	0	0

FIGURE 4.20 – Résultats détaillés du test de charge (interface Locust)

La figure 4.21 illustre l'évolution du débit et des temps de réponse au cours de la montée en charge progressive.

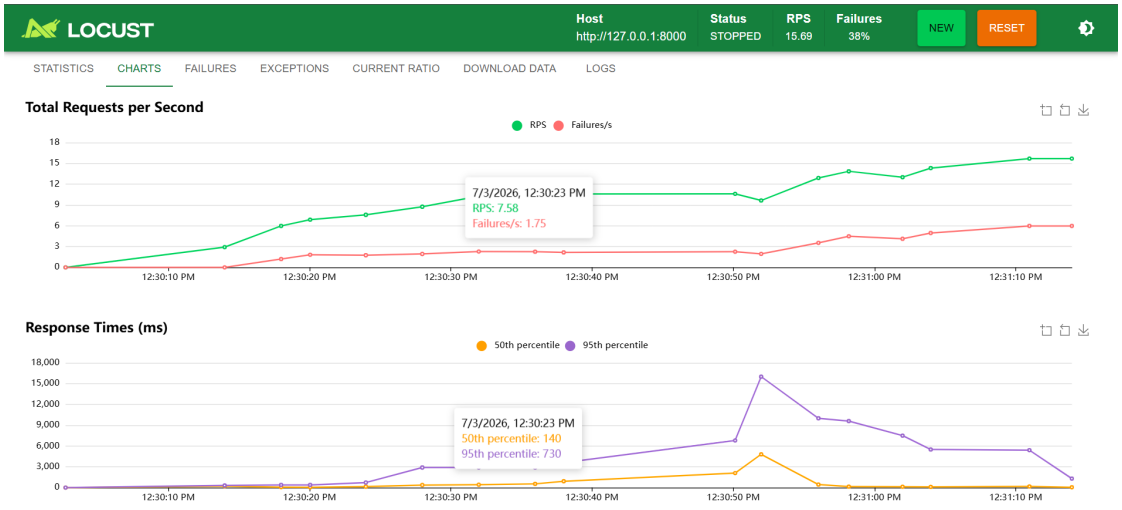


FIGURE 4.21 – Évolution du débit et des temps de réponse pendant la montée en charge

Interprétation des échecs observés : Les échecs sur `/score [invalid]` (422) et `/score [extreme]` (rejet de sérialisation JSON) correspondent au comportement attendu : ces scénarios envoient volontairement des données malformées pour valider la robustesse de la couche de validation, qui rejette correctement 100 % de ces requêtes. Les échecs sur `/score [fraud]` et `/score [legit]` (`429 Too Many Requests`, 41 % et 45 % des requêtes) ne sont pas une défaillance, mais la manifestation du mécanisme de limitation de débit à 200 requêtes/minute (tableau 2.3) : avec 100 utilisateurs simultanés, ce seuil est rapidement dépassé, déclenchant le rejet contrôlé prévu par BNF3.

Limites identifiées : Deux axes d'amélioration se dégagent. L'endpoint `GET /stats/kpis` présente une latence élevée sous charge (médiane de 10 s), probablement due à une agrégation SQL non mise en cache. L'endpoint `/health`, utilisé pour la supervision des conteneurs, ne semble pas isolé de la charge applicative (médiane de 1,3 s sous charge), ce qui pourrait entraîner de faux positifs de disponibilité en production.

Conclusion

Ce chapitre a présenté la mise en production du système FraudGuard, à travers l'API de service développée avec FastAPI et le tableau de bord analytique développé avec React. L'architecture conteneurisée complète permet un déploiement reproductible de l'ensemble des services, complété par une supervision continue (Prometheus, Grafana) incluant le suivi de la dérive du modèle par PSI, ainsi que par des tests de charge validant le comportement du système sous forte concurrence. Ensemble, ces composants permettent aux analystes fraude de superviser l'activité transactionnelle, d'explorer les explications du modèle et d'interagir avec le système en langage naturel.

Conclusion générale et Perspectives

Conclusion générale

La fraude bancaire constitue un enjeu majeur pour les institutions financières, nécessitant des solutions capables de détecter rapidement les comportements suspects tout en restant fiables et explicables. Dans ce contexte, ce projet de fin d'études avait pour objectif de concevoir un système intelligent de détection de fraude en temps réel, combinant performance prédictive, explicabilité et facilité d'utilisation.

Réalisé au sein de **Capgemini Engineering Tunisie**, le système **FraudGuard** a été développé selon la méthodologie IBM, organisée en six phases successives, permettant une progression structurée depuis l'analyse des besoins jusqu'à la mise en œuvre d'une solution complète et déployable.

Sur le plan des données, un pipeline de prétraitement et de feature engineering a été mis en place sur plus de 7,4 millions de transactions synthétiques, conduisant à la création de 52 variables pertinentes tout en garantissant l'absence de fuite d'information grâce à un découpage temporel rigoureux et à une vérification systématique de la causalité des features.

Sur le plan de la modélisation, quatre algorithmes ont été comparés : régression logistique, LightGBM, réseau de neurones et XGBoost, ce dernier étant retenu pour ses performances supérieures et sa compatibilité native avec SHAP. Le modèle final a obtenu d'excellents résultats sur le jeu de test, avec un **ROC-AUC de 0.9852**, un **AUPRC de 0.9577** et un **F1-score de 0.88**, démontrant sa capacité à identifier efficacement les transactions frauduleuses malgré un déséquilibre des classes significatif (ratio 1 : 4). Le seuil de décision optimal (0.6729) a été calibré non pas arbitrairement, mais par minimisation d'un coût métier pondérant davantage les fraudes non détectées que les faux positifs.

L'intégration de SHAP a permis d'apporter une dimension d'explicabilité essentielle en justifiant chaque prédiction à travers l'impact quantifié des variables utilisées par le modèle, à l'échelle globale comme transaction par transaction. Cette transparence renforce la confiance des utilisateurs et répond à l'exigence réglementaire de droit à l'explication.

Enfin, le système a été déployé sous la forme d'une architecture complète intégrant un backend FastAPI, une base de données PostgreSQL, un dashboard React interactif,

un agent conversationnel basé sur Groq, ainsi qu'une stack de monitoring (Prometheus, Grafana, AlertManager) permettant le suivi en temps réel des performances et du comportement du modèle en production.

Au final, FraudGuard démontre qu'il est possible de combiner intelligence artificielle, développement web et outils d'explicabilité afin de construire une solution moderne de détection de fraude répondant aux exigences techniques et opérationnelles du domaine bancaire.

Perspectives

Bien que les objectifs fixés aient été atteints, plusieurs améliorations peuvent être envisagées afin d'enrichir et de faire évoluer la solution proposée.

À court terme, un premier axe d'amélioration consiste à mettre en place un mécanisme de réentraînement automatique du modèle, déclenché lorsque l'indicateur de drift (PSI), déjà suivi via la stack de supervision Grafana (section 4.4.2), dépasse un seuil critique.

Une autre perspective concerne l'affinement de la calibration des seuils de décision : si le seuil principal (0.6729) a été optimisé par coût métier, les bornes définissant les niveaux *Low*, *Medium* et *High* reposent actuellement sur des conventions raisonnables plutôt que sur une optimisation formelle. Une calibration de l'ensemble de ces seuils sur des données de production réelles renforcerait la cohérence du système de décision à quatre niveaux.

L'agent conversationnel intégré au système pourrait également être enrichi par de nouvelles fonctionnalités, comme la génération automatique de rapports d'analyse exportables, la détection proactive de patterns de fraude émergents, ou la création d'alertes intelligentes contextualisées pour les analystes fraude.

À moyen terme, le renforcement de la sécurité constitue un axe important, notamment à travers l'implémentation d'une authentification par jeton (JWT), d'un contrôle d'accès basé sur les rôles (RBAC) et du chiffrement systématique des échanges via HTTPS, en complément des mécanismes de limitation de débit et de masquage des données personnelles déjà en place.

Enfin, à plus long terme, des approches innovantes telles que l'apprentissage fédéré ou les réseaux de neurones sur graphes (*Graph Neural Networks*) pourraient être explorées afin de détecter des schémas de fraude plus complexes, impliquant des réseaux d'acteurs coordonnés que les approches tabulaires classiques ne peuvent pas capturer.

Ces perspectives montrent que FraudGuard constitue une base solide pouvant évoluer vers une plateforme de détection de fraude encore plus performante, intelligente et adaptée aux besoins réels du secteur financier.

Bibliographie

[1] The Nilson Report. *Payment Card Fraud Losses Approach $34 Billion*. GlobeNewswire, 6 janvier 2025. Disponible sur : https://www.globenewswire.com/news-release/2025/01/06/3004931/0/en/Payment-Card-Fraud-Losses-Approach-34-Billion.html (consulté le 02/05/2026).

[2] Parlement européen et Conseil de l'Union européenne. *Règlement (UE) 2016/679 (RGPD) – Article 22 : Décision individuelle automatisée*. Disponible sur : https://eur-lex.europa.eu/legal-content/FR/TXT/HTML/?uri=CELEX:32016R0679#article_22 (consulté le 02/05/2026).

[3] Capgemini. *Qui nous sommes*. Disponible sur : https://www.capgemini.com/fr-fr/qui-nous-sommes/ (consulté le 02/05/2026).

[4] Capgemini Engineering. *Qui nous sommes – Ingénierie et R&D*. Disponible sur : https://www.capgemini.com/fr-fr/ingenierie/ (consulté le 02/05/2026).

[5] Capgemini Engineering Tunisie. *Capgemini Engineering en Tunisie*. Disponible sur : https://www.capgemini.com/fr-fr/pays/tunisie/ (consulté le 02/05/2026).

[6] Ismat Samadov. *Transactions – Realistic Patterns in Financial Transactions for Fraud Detection*. Kaggle. Disponible sur : https://www.kaggle.com/datasets/ismetsemedov/transactions (consulté 22/01/2026).

[7] FICO. *FICO Falcon Fraud Manager – Real-time fraud detection*. Disponible sur : https://www.fico.com/fr/products/fico-falcon-fraud-manager (consulté le 05/05/2026).

[8] Amazon Web Services. *AWS Fraud Detector – Détection de fraude en ligne entièrement gérée*. Disponible sur : https://aws.amazon.com/fr/fraud-detector/ (consulté le 05/05/2026).

[9] IBM. *IBM Data Science Methodology – Approche itérative pour les projets de données*. Disponible sur : https://www.ibm.com/cloud/learn/data-science-methodology (consulté le 05/05/2026).

[10] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning : Data Mining, Inference, and Prediction* (2e éd.). Springer. Disponible sur : https://hastie.su.domains/ElemStatLearn/ (consulté le 08/05/2026).

[11] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM : A Highly Efficient Gradient Boosting Decision Tree. *Advances in Neural Information Processing Systems*, 30, 3146–3154. Disponible sur : https://papers.nips.cc/paper/6907-lightgbm-a-highly-efficient-gradient-boosting-decision-tree(consulté le 08/05/2026).

[12] Chen, T., & Guestrin, C. (2016). XGBoost : A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. Disponible sur : https://dl.acm.org/doi/10.1145/2939672.2939785(consulté le 08/05/2026).

[13] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., ... Chintala, S. (2019). PyTorch : An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems*, 32, 8024–8035. Disponible sur : https://papers.nips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library(consulté le 08/05/2026).

[14] Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems*, 30, 4765–4774. Disponible sur : https://papers.nips.cc/paper/7062-a-unified-approach-to-interpreting-model-predictions(consulté le 15/05/2026)